

AI 기반 업무 자동화 워크플로우 개발

n8n 셀프호스팅으로 만드는 8시간 핸즈온

발주

KOSTA

대상

비개발자 · 기획자 · PM

시간

09:00 – 18:00 (8h)

방식

100% 셀프호스팅

강사 소개

조나래

(주) 드림플로우 대표 / (주) 프렌티스 책임 연구원

- 현) 드림플로우 대표
- 현) (주)프렌티스 AI 플랫폼 연구원
- 전) NH농협 스마트뱅킹 차세대 및 고도화 프로젝트
- 전) 스타트업 서비스 개발 (패션 구독, 여행 플랫폼, IoT 스마트 팩토리)
- 가천대학교 모바일소프트웨어학과 석사

교육 이력

- 한국소프트웨어기술진흥협회 재직자 교육 전문 강사 (5년 이상)
- AI 기반 프론트엔드 개발 및 UI/UX 설계 전문
- Spring, React, LangChain, Claude Code, Supabase, N8N 등 실무 교육 다수

로컬 환경 n8n 기반 AI 자동화 구축

Docker 셀프호스팅 · ngrok 고정 도메인 · OpenAI · Google · Discord · Pinecone

11

Learning Objectives

이론 + 핸즈온

8h

총 교육 시간

오프닝 ~ 클로징

75%

실습 비중

이론 25%

100%

셀프호스팅

클라우드 의존 없음

과정 특징

- 본인 노트북에 Docker 로 n8n 직접 구동
- ngrok 고정 도메인으로 OAuth · Webhook 외부 노출
- AI Agent · Tool · Memory · RAG 까지 한 번에
- Gmail · Discord · Google Drive 실제 연동

이 과정에서 다루지 않는 것

- Kubernetes · 멀티 컨테이너 오케스트레이션
- Cloudflare Named Tunnel · 자체 도메인 운영
- n8n 코드 노드 · 사용자 정의 노드 개발
- 프로덕션 단위의 모니터링 · 가용성 설계

수료 후 갖추게 되는 역량

8시간 과정에서 다루는 6가지 핵심 역량입니다



셀프호스팅

Docker 로 노트북에 n8n 을 실행하고 워크플로우 작성



외부 노출

ngrok 고정 도메인으로 OAuth · Webhook 연결



AI Agent

Chat Trigger · System Prompt · Tool · Memory 조합



실무 통합

Gmail · Discord · Google Drive 실제 계정 연결



RAG 챗봇

Drive 문서를 Pinecone 에 임베딩하고 검색 챗봇으로



운영 감각

데이터 영속 · 백업 · 인증 분리 · 트러블슈팅

11 개 학습 단위 · 4 개 섹션

환경 → 데이터 흐름 → AI 3종 → OAuth/이메일 → 터널/Discord → RAG

Section 1 · 오전 · 환경 + 데이터 흐름 + AI 첫 경험

LO 1 · 15 min
n8n 기초 개념

LO 2 · 25 min
Docker 셀프호스팅

신규

LO 3 · 50 min
데이터 흐름

LO 4 · 50 min
AI Agent 기초

Section 2 · 점심 후 · AI 확장 + OAuth + 첫 자동화

LO 5 · 25 min
AI Tool 활용

LO 6 · 25 min
Memory 시스템

LO 7 · 25 min
Google OAuth

LO 8 · 30 min
AI 이메일 분류

Section 3 · 외부 노출 + Discord

LO 9 · 20 min
ngrok 터널

신규

LO 10 · 60 min
Discord 연동

Section 4 · 클라이맥스

LO 11 · 45 min
Drive 기반 RAG

8 블록 프레임 · 50 분 강의 + 10 분 휴식

09:00 – 18:00 · 점심 12:00 – 13:00

블록	시간	활동	LO
1	09:00 – 09:50	오프닝 + n8n 기초 + Docker 셀프호스팅	LO 1, 2
2	10:00 – 10:50	데이터 흐름 (Data Table · IF/Set · 표현식 · Loop)	LO 3
3	11:00 – 11:50	AI Agent 기초 (Chat Trigger · System Prompt)	LO 4
—	12:00 – 13:00	점심	
4	13:00 – 13:50	AI Tool 활용 + Memory 시스템	LO 5, 6
5	14:00 – 14:50	Google OAuth + AI 이메일 분류	LO 7, 8
6	15:00 – 15:50	ngrok 터널 + Discord 단방향	LO 9, 10 전반
7	16:00 – 16:50	Discord 양방향 + 통합 점검	LO 10 후반
8	17:00 – 17:50	Drive 기반 RAG + 클로징	LO 11

수강 전 준비 항목

대부분 무료, 강의 시작 전 안내된 가입 단계만 완료하면 됩니다



노트북

Docker Desktop 설치 가능 (mac/Windows, RAM 8GB+)

Docker 가 동작하는 환경이면 충분



n8n 이미지

n8nio/n8n:2.20.9

2.19 회귀 2건(챗 응답 #295678 · OpenAI axios 헤더 #29471) 모두 해소된 안정 버전 — 태그 고정 필수



ngrok 무료 계정

Authtoken 1개 + dev domain 1개 (*.ngrok-free.dev, 기존 계정은 .app 유지)

도메인 고정 — OAuth redirect 재등록 불필요



OpenAI API Key

강사 공용 키 배포 (npx claude-api-setup 패턴)

수강생 별도 결제 부담 없음



Google 계정

Gmail · Drive 활성, OAuth Client 생성 가능

Google Cloud Console 콘솔 사용



Discord 계정

테스트용 서버 + 채널 1개

n8n trigger 앱으로 양방향 봇 구축



Pinecone

무료 Starter 티어

RAG 벡터 인덱스 1개



SerpAPI

월 250회 무료 플랜

AI Tool 의 웹 검색 데모용

01

n8n 기초 개념

- ✓ n8n의 목적과 활용 분야를 설명할 수 있습니다
- ✓ 워크플로우, 트리거, 노드, 아이템, 실행 개념을 이해합니다
- ✓ n8n 인터페이스를 탐색할 수 있습니다
- ✓ 첫 워크플로우를 실행할 수 있습니다

🕒 30분

이번 장에서 배우는 내용

- n8n이란 무엇인지 알아봅니다
- 핵심 용어 5가지를 학습합니다
- n8n과 Zapier, Make를 비교합니다
- cloud와 셀프호스팅의 차이를 이해합니다
- 인터페이스를 직접 탐색합니다
- 첫 워크플로우를 만들고 실행합니다



**노드를 연결해서
자동화를 만듭니다**

n8n 플랫폼 개요

노코드/로우코드 워크플로우 자동화 플랫폼입니다

- 시각적 캔버스에서 노드를 연결하여 업무 자동화를 구현합니다
- 500개 이상의 서비스 연동과 AI Agent 기능을 내장하고 있습니다
- fair-code 라이선스 기반 오픈소스로 셀프호스팅이 가능합니다
- LangChain/LangGraph 프레임워크 기반의 AI 기능을 제공합니다
- 코드 노드(Python/JS)로 유연하게 확장할 수 있습니다

500+

연동 서비스

공식 노드 기준

AI

Agent 내장

LangChain 기반

OSS

오픈소스

fair-code 라이선스

핵심 용어 5가지

n8n을 사용하기 전에 반드시 알아야 할 개념입니다



워크플로우

자동화 시나리오 전체를 담는 캔버스입니다 (도화지)



트리거

워크플로우를 시작하는 신호입니다 (시작 버튼)



노드

워크플로우를 구성하는 작업 단위입니다 (레고 블록)



아이템

노드 사이를 흐르는 JSON 데이터 단위입니다 (택배 상자)



실행

워크플로우가 실제로 돌아간 한 번의 기록입니다 (배송 이력)

실행 순서 원칙

워크플로우 내에서 노드가 실행되는 규칙입니다



왼쪽에서 오른쪽으로

데이터는 연결된 선을 따라 한 노드의 출력에서 다음 노드의 입력으로 이동합니다



위에서 아래로

하나의 노드에서 여러 선이 나갈 때, 위쪽 경로가 아래쪽 경로보다 먼저 실행됩니다



모든 입력 대기

여러 선이 하나의 노드로 들어올 때, 모든 데이터를 받을 때까지 대기합니다

n8n vs Zapier vs Make

업무 자동화 도구별 핵심 비교입니다

항목	n8n	Zapier	Make
가격	셀프호스팅 무료, Cloud Starter 월 €24부터	유료 월 \$19.99부터	유료 월 \$9부터
연동 서비스	500개 이상	2,000개 이상	1,000개 이상
AI 기능	AI Agent 내장, LangChain 기반	GPT 연동 수준	제한적
커스터마이징	코드 노드(Python/JS) 자유 확장	제한적	중간 수준
셀프호스팅	가능 (Docker)	불가	불가
적합 대상	AI 자동화, 개발팀	간단한 업무 자동화	비개발자, 콘텐츠

cloud vs 셀프호스팅

이 과정에서는 Docker 기반 셀프호스팅 버전을 사용합니다

항목	n8n.cloud	셀프호스팅
설치	가입 즉시 사용	서버에 직접 설치 필요
비용	Starter 월 €24부터	서버 비용만 (월 5~20유로)
실행 제한	Starter 기준 월 2,500 executions	제한 없음
관리	n8n이 관리	직접 업데이트, 백업 관리
적합 대상	빠른 시작, 비개발자	비용 절감, 보안 요구

학습 및 프로토타입에는 cloud, 운영 환경에서는 셀프호스팅을 고려할 수 있습니다.

인터페이스 투어: 뷰 모드 전환

중앙 상단에서 워크플로우의 작업 단계를 전환합니다

- Editor: 노드를 배치하고 연결하며 워크플로우를 직접 설계하는 공간입니다
- Executions: 과거 실행 이력(로그)을 확인합니다. 성공/실패 여부와 각 노드의 데이터를 볼 수 있습니다
- Evaluations: AI나 테스트 시나리오를 설정하여 워크플로우 성능을 평가하는 모드입니다

모드	용도	확인 가능한 정보
Editor	워크플로우 설계 및 편집	노드 배치, 연결, 설정값
Executions	실행 이력 확인	성공/실패, 각 노드 입출력 데이터
Evaluations	성능 평가 및 테스트	테스트 결과, 정확도 분석

인터페이스 투어: 상단 도구 모음

캔버스 상단에 위치한 5가지 핵심 도구입니다

아이콘	명칭	역할
+	Nodes panel	노드 라이브러리를 열어 캔버스에 추가합니다
검색	Command bar (Ctrl+K)	노드 검색, 저장, 실행 등 모든 명령을 텍스트로 입력합니다
메모	Sticky note	캔버스에 메모장을 생성하여 주석을 남깁니다
패널	Side panel	선택한 노드의 데이터를 워크플로우와 동시에 확인합니다
AI	n8n AI	자연어로 워크플로우 생성을 요청하거나 에러 수정을 돕습니다

핵심 단축키. Command bar(Ctrl+K)는 n8n에서 가장 자주 사용하는 기능입니다. 노드 검색, 워크플로우 전환, 실행까지 모두 여기서 가능합니다.

인터페이스 투어: 하단 도구 및 Output 뷰

아이콘	명칭	설명
[]	Zoom to fit	워크플로우 전체가 화면에 맞도록 확대/축소를 자동 조절합니다
정렬	Tidy up	흩어진 노드들을 보기 좋게 자동 정렬합니다. 선이 꼬였을 때 유용합니다
+	Zoom in	캔버스를 확대하여 노드의 상세 연결 상태를 확인합니다
-	Zoom out	캔버스를 축소하여 전체 워크플로우 흐름을 파악합니다

Output 뷰 3가지



Schema

필드 구조를 트리 형태로 확인합니다



Table

데이터를 표 형태로 비교합니다



JSON

원시 JSON 데이터를 확인합니다

노드 분류 — 큰 두 갈래와 AI용 서브노드

공식 분류: Trigger / Action. AI Agent에는 Model · Tool · Memory 서브노드를 부착합니다



Trigger

워크플로우의 시작점입니다. Manual, Schedule, Chat, Webhook 등이 있습니다



Action

결과를 만드는 모든 노드입니다. Set, IF, Merge, Gmail, HTTP Request뿐 아니라 AI Agent·OpenAI Chat Model도 Action에 속합니다



Cluster 서브노드 — Model · Memory

AI Agent에 연결하는 LLM과 대화 기억. OpenAI/Anthropic/Ollama Chat Model, Simple Memory(구 Window Buffer Memory) 등이 있습니다



Cluster 서브노드 — Tool

AI Agent가 판단하여 호출하는 도구입니다. Wikipedia, Calculator, Gmail Tool, Vector Store QA Tool 등이 있습니다

JSON 데이터 기초

n8n에서 데이터는 JSON 형태로 노드 사이를 흐릅니다

```
n8n 아이템 구조 json
[
  {
    "json": {
      "이름": "김철수",
      "이메일": "kim@example.com",
      "부서": "기획팀"
    }
  }
]
```

표현식으로 데이터 참조

표현식	결과
{{ \$json.이름 }}	"김철수"
{{ \$json.이메일 }}	"kim@example.com"
{{ \$json.부서 }}	"기획팀"

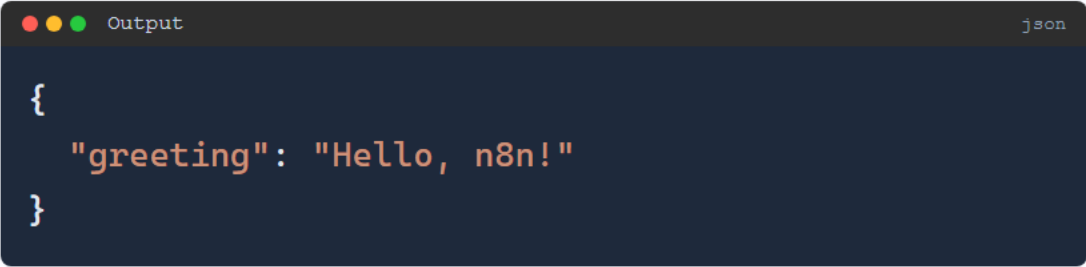
실습: 첫 워크플로우 만들기

Manual Trigger와 Set 노드를 연결하여 실행합니다

실습 단계

- 1 Workflows에서 Add Workflow 클릭
- 2 워크플로우 이름을 "첫 번째 테스트"로 변경
- 3 Manual Trigger 추가
- 4 우측 + 버튼으로 Set 노드 추가
- 5 Name: greeting, Value: "Hello, n8n!"
- 6 Test Workflow 버튼으로 실행
- 7 Set 노드의 Output 탭에서 결과 확인

예상 결과

A screenshot of the n8n 'Output' window. The window has a dark background and a title bar with three colored dots (red, yellow, green) and the text 'Output'. In the top right corner, the word 'json' is displayed. The main area shows a JSON object:

```
{  "greeting": "Hello, n8n!"}
```

n8n 기초 개념 체크포인트

- ✓ n8n과 Zapier/Make의 주요 차이점을 설명할 수 있습니다
- ✓ 워크플로우, 트리거, 노드, 아이템, 실행 개념을 이해했습니다
- ✓ Trigger와 Action의 구분, 그리고 AI Agent용 Cluster 서브노드(Model/Tool/Memory)를 설명할 수 있습니다
- ✓ Manual Trigger와 Set 노드로 첫 워크플로우를 실행했습니다
- ✓ {{ \$json.field }} 표현식의 의미를 이해했습니다

02

Docker로 n8n 셀프호스팅

- ✓ Docker Desktop 설치 및 정상 실행 상태 확인
- ✓ 이미지, 컨테이너, 볼륨의 역할과 차이 이해
- ✓ docker compose로 n8n 컨테이너를 기동하고 localhost:5678 접속 확인
- ✓ 운영 명령어(compose up -d, ps, logs, compose down) 숙달
- ✓ n8n_data 폴더가 컨테이너 삭제 후에도 유지됨을 확인

🕒 25분 · 블록 1 후반

Docker — 컨테이너 기반 오픈 플랫폼

코드와 의존성을 하나의 단위로 묶어 어떤 환경에서도 동일하게 실행할 수 있습니다

Docker 는 애플리케이션을 코드 · 의존성 · 런타임과 함께 **컨테이너(container)** 라는 격리된 단위로 패키징하여, 어떤 환경에서도 동일하게 실행할 수 있게 해 주는 오픈 플랫폼입니다.

출처. docs.docker.com — *Docker is an open platform for developing, shipping, and running applications in isolated environments called containers.*



패키징

코드 · 라이브러리 · 런타임 · 시스템 설정을 하나의 이미지로 묶어 배포 단위로 사용합니다



격리

각 컨테이너는 호스트 OS · 다른 컨테이너로부터 분리된 자체 파일시스템 · 네트워크 · 프로세스 공간을 가집니다



이식성

같은 이미지가 로컬 노트북 · 사내 서버 · 클라우드에서 동일하게 동작해, 환경 차이로 인한 버그를 없앱니다

n8n Cloud와 셀프호스팅의 차이

본 과정은 셀프호스팅을 채택하였으며, 추가 비용 없이 무제한 실행이 가능하고 데이터 외부 유출이 없습니다

항목	n8n.cloud	셀프호스팅 (이 과정)
시작	가입 후 즉시	Docker 설치 후 명령어 한 줄
비용	Starter 월 €24부터	무료
실행 제한	Starter 기준 월 2,500 executions	무제한
데이터 위치	n8n 서버	본인 컴퓨터
적합 대상	빠른 시작	비용 절감 · 보안 요구 · 학습

핵심. 본인 노트북에 Docker 로 n8n 을 띄워 8시간 동안 실습합니다. 클라우드 가입 없이도 같은 노드와 워크플로우를 씁니다.

Docker와 VM의 자원 사용 차이

호스트 커널을 공유하기 때문에 부팅이 빠르고 메모리 사용량이 작습니다

항목	VM (가상머신)	Docker (컨테이너)
OS	통째로 새로 설치 (GB)	호스트 커널 공유 (MB)
부팅	분 단위	초 단위
동시 실행	2 - 3 개	수십 개
격리 수준	강함	적당함
이 과정의 용도	사용 안 함	n8n · cloudflared · ngrok 모두 컨테이너

Docker Desktop 은 내부적으로 작은 리눅스 VM 을 자동으로 실행하고 그 안에서 컨테이너를 구동합니다. 사용자가 직접 관리할 필요는 없으며, 메뉴바 고래 아이콘, Settings, Resources 경로에서 메모리 할당만 한 번 확인하면 충분합니다.

-p 호스트포트 : 컨테이너포트

호스트와 컨테이너를 연결하는 양방향 포트 번호. 양쪽이 달라도 됩니다



예: 호스트 8080 번 요청을 컨테이너 5678 번으로 전달하려면 -p 8080:5678

docker-compose 설정 파일

셀프호스팅 단계의 최소 구성으로 n8n 단일 서비스만 포함하며, 외부 노출은 LO-9에서 추가합니다

```
services:
  n8n:
    image: n8nio/n8n:2.20.9          # 태그 고정 - 2.19 회귀 2건(#295678, #29471) 해소된 안정 버전
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"                 # 호스트 : 컨테이너
    environment:
      - GENERIC_TIMEZONE=Asia/Seoul
      - TZ=Asia/Seoul
    volumes:
      - ./n8n_data:/home/node/.n8n # 영속 데이터 - Finder 에서 직접 열림
```

강의 레포 / docker-compose.yml

주의 — 버전 고정. 2.19 계열은 챗 응답 먹통 회귀(이슈 #295678)와 `api.openai.com` OpenAI 호출 axios 헤더 회귀(이슈 #29471)가 함께 있어, 두 회귀가 모두 해소된 **n8nio/n8n:2.20.9** 로 고정합니다. `:latest` 금지.

필수 운영 명령

모든 명령어는 컨테이너 이름 'n8n'을 기준으로 하며, docker-compose.yml의 container_name과 동일합니다

명령어	용도
<code>docker compose up -d</code>	compose 파일 기준 컨테이너 백그라운드 기동
<code>docker compose down</code>	compose 로 띄운 컨테이너 일괄 중지 및 제거
<code>docker ps</code>	실행 중인 컨테이너 목록 확인
<code>docker logs n8n</code>	n8n 컨테이너 로그 확인 (-f 옵션으로 실시간 추적)

데이터 안전성. docker compose down, 컨테이너 삭제, 컴퓨터 재부팅, Docker Desktop 종료 — 모두 데이터는 안전합니다. 데이터가 사라지는 동작은 `n8n_data/` 폴더를 직접 삭제하는 것입니다.

로컬 환경에서 n8n 실행

강의 레포 폴더 내에서 5단계로 진행됩니다

- 1 Docker Desktop 설치** 5 min
docker.com 에서 OS 별 설치 → 메뉴바 고래 아이콘 확인 → `docker --version`
- 2 강의 레포 받기 & 이미지 다운로드** 5 min
`git clone n8n-workflow-kosta-260521` → `docker pull n8nio/n8n:2.20.9`
- 3 컨테이너 기동** 3 min
`docker compose up -d` → 브라우저에서 `localhost:5678` 접속 → 회원가입
- 4 운영 명령어 따라치기** 5 min
`docker compose up -d` · `ps` · `logs` · `compose down` 을 순서대로 실행
- 5 데이터 영속 확인** 2 min
`docker compose down` → `up` 후에도 가입 정보와 워크플로우가 유지됨을 확인

자주 발생하는 다섯 가지 오류

실습 중 빈번하게 발생하는 오류와 핵심 해결책을 정리했습니다

증상	원인	해결
docker: command not found	Docker Desktop 미설치 또는 미실행	Docker Desktop 실행 후 재시도
port already in use	5678 포트가 이미 사용 중	-p 5679:5678 로 다른 포트 매핑
컨테이너가 즉시 종료	이미지 다운로드 실패 또는 메모리 부족	docker logs n8n 로 원인 확인 · 메모리 늘리기
브라우저 접속 안 됨	컨테이너가 떠 있지 않음	docker ps 확인 → docker start n8n
OpenAI 호출 시 config.headers.setContentType (#29471) is not a function 또는 챗 응답 먹 통	n8n 2.19 회귀 2건 — 챗 응답(#295678) · OpenAI axios 헤더	docker exec n8n n8n --version 으로 버전 확인 → 2.19.x 이면 컨테이너 삭제 후 n8nio/n8n:2.20.9 재실행

Docker 셀프호스팅 체크포인트

- ✓ `docker --version` 으로 Docker Desktop 정상 실행 확인
- ✓ `docker compose up -d` 로 n8n 컨테이너 기동
- ✓ `http://localhost:5678` 에서 회원가입 또는 로그인 성공
- ✓ `docker compose up -d · ps · logs · compose down` 직접 실행
- ✓ 컨테이너 삭제 후 다시 실행해도 가입 정보가 유지됨을 확인

03

데이터 흐름

- ✓ 표현식으로 데이터를 참조할 수 있습니다
- ✓ IF 노드로 조건 분기를 구현할 수 있습니다
- ✓ Set/Merge 노드로 데이터를 변환하고 병합할 수 있습니다
- ✓ 실행 로그를 확인하여 디버깅할 수 있습니다

🕒 30분

이번 장에서 배우는 내용

pageNumber=33totalPages=148

- 표현식(Expression)으로 데이터를 참조합니다
- IF 노드로 조건 분기를 구현합니다
- Set 노드로 데이터를 변형합니다
- Merge 노드로 분기된 데이터를 병합합니다
- Loop 처리 패턴을 학습합니다
- 실행 로그를 확인하여 디버깅합니다



데이터를 분기하고
다시 합칩니다

Data Table: n8n 내장 데이터베이스

외부 서비스 없이 n8n 안에서 데이터를 저장하고 조회할 수 있습니다

Data Table이란

- n8n에 내장된 간단한 데이터베이스입니다
- 외부 DB나 스프레드시트 없이도 데이터를 저장하고 관리합니다
- UI에서 직접 행을 추가/수정/삭제할 수 있습니다
- 워크플로우에서 노드로 읽기/쓰기가 가능합니다



직접 입력

UI에서 엑셀처럼 데이터를 입력합니다



워크플로우에서 읽기

Data Table 노드로 데이터를 가져옵니다



워크플로우에서 쓰기

RSS, API 등의 결과를 자동 저장합니다

실습: Data Table에 샘플 데이터 입력

좌측 사이드바 Data Tables에서 새 테이블을 만들고 4건의 데이터를 입력합니다

입력할 데이터 (업무 요청 목록)

이름	부서	요청	긴급
김철수	기획팀	휴가 신청	true
이영희	개발팀	장비 구매	false
박민수	기획팀	출장 보고	true
최지은	마케팅팀	예산 검토	false

생성 방법. 좌측 사이드바 Data Tables 메뉴에서 "New Data Table"을 선택하고, 테이블 이름을 "업무요청"으로 지정한 후 위 4건을 직접 입력합니다.

다음 단계. 이 데이터를 워크플로우에서 읽어와 IF(긴급 분기), Set(상태 추가), Merge(합치기)를 연습합니다. 외부 서비스 없이 데이터 흐름에 집중합니다.

Data Table 조회 옵션

Get 연산에서 사용할 수 있는 필터, 정렬, 제한 옵션입니다

옵션	설명
Conditions	조건에 맞는 행만 필터링 (Any/All)
Return All	ON이면 조건에 맞는 모든 행 반환
Limit	Return All OFF 시 최대 행 수 (기본 50)
Order By	정렬 기준 컬럼과 방향 지정

실습 팁. Return All ON으로 전체 데이터를 가져옵니다. 실무에서는 Conditions와 Limit 을 함께 사용하십시오.

Conditions 연산자

연산자	설명
Equals	값이 같으면 통과합니다
Not Equals	값이 다르면 통과합니다
Contains	값을 포함하면 통과합니다
Greater Than	값이 크면 통과합니다
Is Empty	값이 비어있으면 통과합니다

IF 노드: 조건 분기

조건에 따라 데이터 흐름을 두 갈래로 나눕니다

- 조건 충족 데이터는 true 경로(위쪽)로 분기합니다
- 조건 미충족 데이터는 false 경로(아래쪽)로 분기합니다
- String, Number, Boolean, Date 조건 유형을 지원합니다
- AND/OR 복합 조건을 조합할 수 있습니다

분기 흐름

IF 노드

true: 조건 충족

false: 조건 미충족

IF 노드 주요 연산자

조건 유형에 맞는 연산자를 선택하여 사용합니다

연산자	설명	예시
Contains	특정 텍스트 포함 여부	"회의 안건" contains "회의"
Equal	정확히 일치	"high" equal "high"
Starts With	특정 텍스트로 시작	"[공지] 안내" starts with "[공지]"
Greater Than	숫자가 큰 경우	100 greater than 50
Regex	정규식 패턴 매칭	"ABC-123" regex "[A-Z]+-\d+"

! 숫자 비교를 String 타입으로 설정하면 타입 불일치 오류가 발생합니다

Set 노드: 데이터 변환

데이터에 새 필드를 추가하거나 기존 필드를 수정합니다

- Add Field를 클릭하여 필드를 추가합니다
- Name과 Value를 입력합니다
- Value에 표현식으로 동적 값을 설정할 수 있습니다

Include 설정	결과
OFF	새 필드만 출력 (기존 데이터 사라짐)
ON, All Fields	기존 필드 유지 및 새 필드 추가

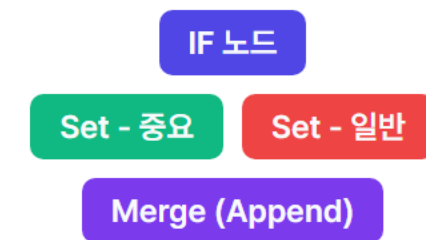
Include Other Input Fields를 ON으로 설정해야 기존 데이터가 유지됩니다. 가장 흔한 실수입니다.

Merge 노드: 데이터 병합

여러 경로로 분기된 데이터를 다시 하나로 합칩니다

- IF 노드로 분기된 데이터를 하나의 흐름으로 다시 통합할 때 사용합니다
- 캔버스에 직접 추가한 후 양쪽 노드의 출력을 수동으로 연결합니다
- 노드 이름을 역할에 맞게 변경하면 가독성이 향상됩니다
- 노드 수가 10개를 넘어가면 이름 규칙이 필수적입니다

IF 분기 후 Merge 패턴



Merge 노드 병합 모드

상황에 따라 적절한 병합 모드를 선택합니다

모드	동작	사용 사례
Append	두 입력 데이터를 순서대로 연결합니다	IF 분기 후 단일 흐름으로 통합
Combine	같은 위치의 아이템끼리 필드를 합칩니다	두 소스의 데이터 통합
Choose Branch	특정 입력만 선택합니다	조건에 따라 한쪽만 사용

실무 선택 가이드. IF 분기 결과를 다시 합칠 때는 **Append**, 두 다른 소스 (예: DB-API)의 데이터를 한 행으로 합칠 때는 **Combine**을 사용합니다.

주의. Combine 모드는 같은 인덱스 위치의 아이템끼리 합치므로, 두 입력의 행 수와 정렬이 맞아야 의도한 결과가 나옵니다.

표현식(Expression) 기초

노드 간 데이터를 연결할 때 표현식을 사용합니다

표현식	설명	결과 예시
<code>{{ \$json.subject }}</code>	현재 아이템의 subject 필드	"회의 안건"
<code>{{ \$json.from }}</code>	현재 아이템의 from 필드	"kim@example.com"
<code>{{ \$json.subject ?? '제목 없음' }}</code>	필드가 없을 때 기본값 지정	"제목 없음"
<code>{{ \$('Set').item.json.name }}</code>	특정 노드의 데이터 참조	"김철수"

- Fixed/Expression 토글로 모드를 전환합니다
- {{ 입력 시 자동으로 표현식 모드가 활성화됩니다
- Input 패널에서 드래그 앤 드롭으로 표현식을 삽입할 수 있습니다

Loop 처리 패턴

n8n에서 반복 처리를 구현하는 방법입니다

- n8n은 기본적으로 모든 아이템을 자동 반복 처리합니다
- Data Table에서 4건을 가져오면 다음 노드에서 4번 실행됩니다
- Loop Over Items 노드로 명시적 반복을 구현할 수 있습니다
- 배치 처리가 필요한 경우 Split In Batches를 사용합니다

점진적 빌드 개발 패턴

Trigger를 추가하고 실행하여 출력을 확인한 후, 다음 노드를 추가하고 다시 테스트하는 과정을 반복합니다. 전체를 한꺼번에 구성하면 문제 원인을 특정하기 어렵습니다.

실행 로그 확인법

워크플로우 디버깅의 핵심은 각 노드의 Output 탭 추적입니다

확인 항목	방법	활용
노드 출력 데이터	노드 클릭 후 Output 탭	데이터 형태 확인
실행 성공 여부	Executions 메뉴 색상 표시	오류 노드 특정
IF 분기 결과	IF 노드 true/false 아이템 수	조건식 검증
표현식 결과	Expression 탭 미리보기	표현식 오류 사전 확인
하단 로그 패널	실시간 노드 상태 표시	병목 노드 파악

! IF 노드 포함 워크플로우에서는 true와 false 양쪽 경로를 모두 테스트해야 합니다

실습: 조건 분기와 병합 워크플로우

Manual Trigger에서 시작하여 IF 분기, Set 가공, Merge 통합을 구성합니다

실습 단계

- "데이터 흐름 실습" 워크플로우 생성
- Manual Trigger, Set(샘플 데이터) 추가
- IF 노드: {{ \$json.subject }} Contains "중요"
- true 경로: Set - 중요 (priority = "high")
- false 경로: Set - 일반 (priority = "low")
- Merge 노드(Append 모드)로 두 흐름을 하나로 통합
- Test Workflow 실행 후 결과 확인

확인 포인트

- ✓ IF 노드에서 데이터가 true 경로로 분기되었는지 확인합니다
- ✓ Set 노드에서 priority 필드가 추가되었는지 확인합니다
- ✓ Include Other Input Fields가 ON인지 확인합니다
- ✓ Merge 노드에서 양쪽 데이터가 하나의 흐름으로 통합되었는지 확인합니다

데이터 흐름 체크포인트

- ✓ 표현식 {{ \$json.field }}를 사용하여 데이터를 참조할 수 있습니다
- ✓ IF 노드에서 조건에 따라 데이터가 분기됩니다
- ✓ Set 노드에서 새 필드가 추가되고 기존 필드가 유지됩니다
- ✓ Merge 노드에서 양쪽 데이터가 합쳐집니다
- ✓ Executions에서 실행 로그를 확인하여 데이터 흐름을 추적할 수 있습니다

04

AI Agent 기초

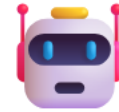
- ✓ Chat Trigger로 대화형 인터페이스를 만들 수 있습니다
- ✓ AI Agent 노드의 구조와 역할을 설명할 수 있습니다
- ✓ OpenAI Chat Model을 연결할 수 있습니다
- ✓ System Prompt를 작성하여 AI 행동을 제어할 수 있습니다

🕒 30분

이번 장에서 배우는 내용

pageNumber=48totalPages=148

- Chat Trigger로 대화형 인터페이스를 만듭니다
- AI Agent 아키텍처를 이해합니다
- OpenAI Chat Model을 연결합니다
- System Prompt를 작성합니다
- 비결정적 동작의 특성을 이해합니다
- 첫 AI 챗봇을 만들고 테스트합니다



**AI Agent가
대화를 처리합니다**

Chat Trigger

n8n에 내장된 채팅 UI를 제공하는 트리거 노드입니다

사용 방법

- 캔버스 더블클릭 후 "chat" 검색하여 추가합니다
- 워크플로우 저장 후 하단 Chat 버튼으로 테스트합니다
- 활성화 시 Webhook URL로 외부 서비스 연동이 가능합니다
- Make Chat Publicly Available로 공개 채팅 URL을 생성합니다



내장 채팅 UI

테스트 시 n8n 내장 채팅 UI를 사용합니다

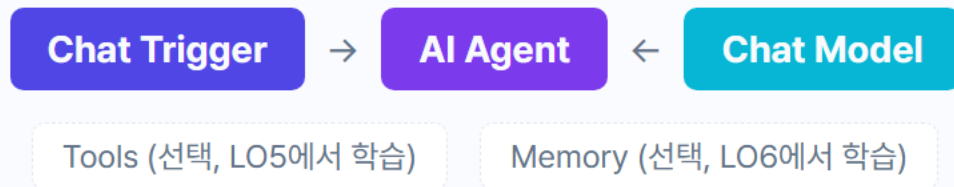


세션 리셋

New Session 버튼으로 대화를 초기화합니다. 워크플로우 전환 시 필수입니다

AI Agent 아키텍처

세 가지 필수 구성요소와 두 가지 선택 구성요소로 이루어집니다



Chat Trigger

사용자의 대화 입력을 받는 시작점입니다. 채팅 UI를 자동으로 제공합니다.

AI Agent

LLM에게 역할을 부여하고, Tool과 Memory를 관리하는 중심 노드입니다.

Chat Model

실제 텍스트를 생성하는 LLM입니다. GPT-4o, Claude 등을 연결합니다.

AI Agent 구성요소 상세

각 구성요소의 역할과 연결 유형을 확인합니다

구성요소	역할	필수	연결 표시
Chat Trigger	사용자 입력을 받는 대화 시작점	필수	회색 실선
AI Agent	LLM과의 통신을 관리하는 중앙 허브	필수	—
Chat Model	실제 AI 모델 연결 (OpenAI 등)	필수	보라색 점선
Tools	AI가 필요할 때 호출하는 도구	선택	노란색 점선
Memory	대화 기록을 유지하는 기억 장치	선택	파란색 점선

OpenAI Chat Model 연동

AI Agent에 Chat Model을 연결하여 AI 응답을 생성합니다

연결 단계

- AI Agent 노드 클릭
- 하단 + Chat Model 클릭
- OpenAI Chat Model 선택
- Create New Credential 클릭
- 강사 제공 API 키 입력 후 Save
- 모델 선택: gpt-5-nano

모델	특징	권장 용도
gpt-5-nano	빠른 응답, 최저 비용	학습, 프로토타입 (이 과정 권장)
gpt-4o-mini	안정적 성능, 저렴	안정화된 워크플로우
gpt-4o	높은 품질, 고비용	복잡한 판단 작업

System Prompt 작성

AI의 역할, 규칙, 제약 조건을 정의합니다

```
System Prompt 예시
```

```
당신은 친절한 업무 도우미입니다.
```

```
규칙:
```

```
1. 사용자의 질문에 친절하게 답변하십시오
```

```
2. 답변은 500자 이내로 간결하게 작성하십시오
```

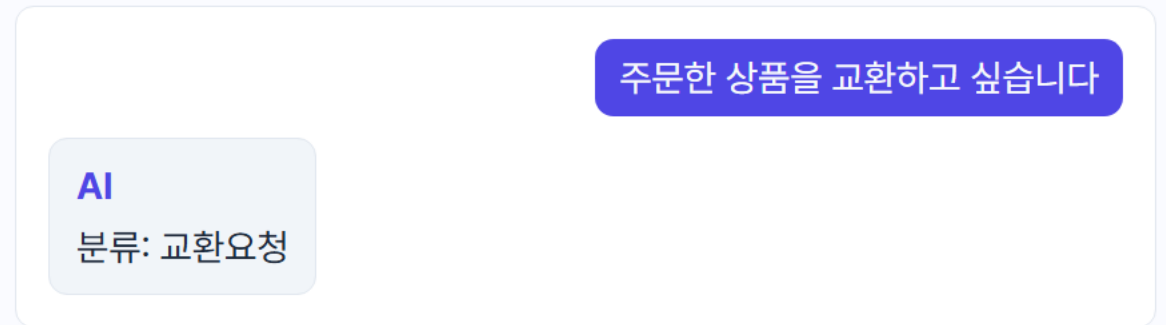
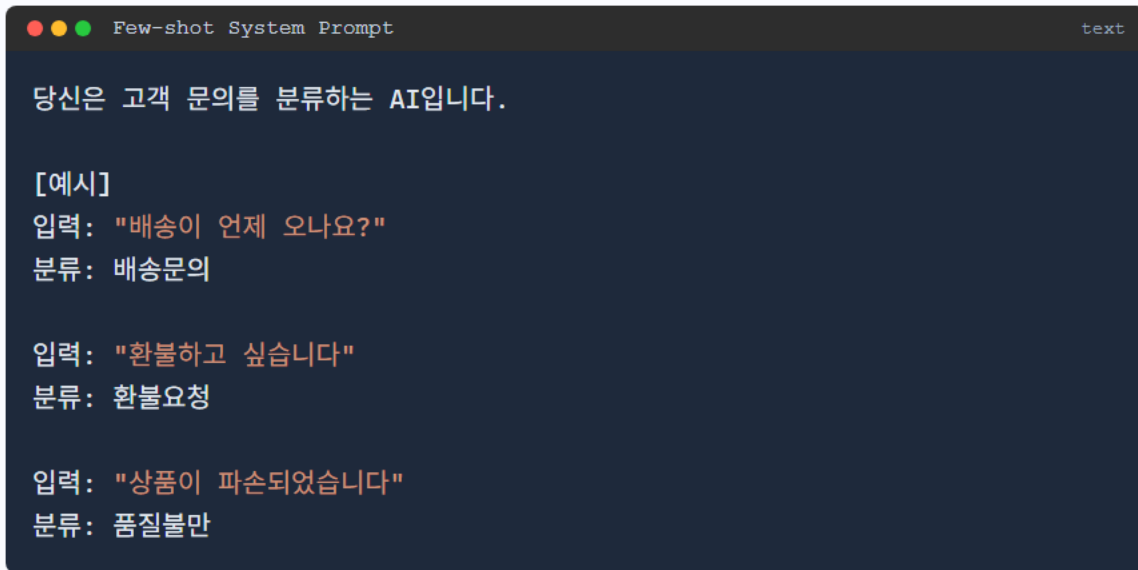
```
3. 모르는 내용은 솔직하게 모른다고 말하십시오
```

```
4. 한국어로 답변하십시오
```

요소	설명
역할(Role)	"당신은 고객 상담원입니다"
규칙(Rules)	"항상 존댓말을 사용하십시오"
제약 (Constraints)	"200자 이내로 답변하십시오"
출력 형식 (Format)	"마크다운 목록으로 출력하십시오"

Few-shot 프롬프트 기법

구체적인 입출력 예시를 포함하면 AI의 응답 품질이 향상됩니다



비결정적 동작의 특성

AI Agent는 일반 소프트웨어와 근본적으로 다릅니다

구분	일반 소프트웨어	AI Agent
동일 입력	항상 동일 결과	매번 다를 수 있습니다
규칙 준수	코드대로 100% 실행	System Prompt를 간헐적으로 무시합니다
디버깅	재현 가능	동일 조건에서 재현이 불가할 수 있습니다

! 중요한 비즈니스 로직은 LLM에 위임하지 않아야 합니다

! Tool을 연결하면 회색지대(Hallucination 위험)를 축소할 수 있습니다

실습: 대화형 챗봇 만들기

Chat Trigger, AI Agent, OpenAI Chat Model을 연결합니다

실습 단계

- "AI 챗봇 테스트" 워크플로우 생성
- Chat Trigger 추가
- Chat Trigger 우측에서 AI Agent 추가
- AI Agent 하단 + Chat Model로 OpenAI Chat Model 추가
- Credential 생성 후 강사 제공 API 키 입력
- 모델: gpt-5-nano 선택
- System Message에 실습용 프롬프트 입력
- 워크플로우 저장 (Ctrl+S)
- 하단 Chat 버튼으로 테스트

테스트 질문 예시

→ "안녕하세요"

→ "n8n이 무엇인지 간단히 설명해 주세요"

→ "오늘 할 일 목록을 만들어 주세요"

실습 결과 확인 및 프로덕션 한계

Executions 로그에서 메시지 구조를 확인하고 한계를 이해합니다

로그에서 확인할 메시지 유형

유형	역할
System	System Prompt에 작성한 AI 행동 규칙
Human	채팅 UI에서 사용자가 입력한 메시지
AI	LLM이 생성한 답변

프로토타입의 한계

- ! Memory 없이는 매 턴마다 이전 대화를 잊습니다
- ! Tool 없이는 학습 데이터에만 의존합니다
- ! 비용 제어, 인증, 모니터링이 없습니다
- ! 에러 처리(타임아웃 등)가 없습니다

AI Agent 기초 체크포인트

- ✓ Chat Trigger 노드를 추가하고 채팅 UI를 열 수 있습니다
- ✓ AI Agent에 OpenAI Chat Model이 정상 연결됐습니다 (보라색 점선)
- ✓ System Prompt를 설정하고 AI가 그에 따라 응답합니다
- ✓ 채팅 세션을 리셋하고 새로운 대화를 시작할 수 있습니다
- ✓ Executions 로그에서 System/Human/AI 메시지 구조를 확인할 수 있습니다

05

AI Tool 활용

- ✓ Tool의 개념과 역할을 이해하고 설명할 수 있습니다
- ✓ Wikipedia, Calculator, SerpAPI 등 내장 Tool을 연결할 수 있습니다
- ✓ 여러 Tool을 연결하여 AI가 자동 선택하는 패턴을 확인합니다
- ✓ Tool 과다 문제와 멀티 에이전트 패턴을 이해합니다

🕒 30분

AI Agent와 Tool 연동

Tool은 AI가 사용하는 외부 기능입니다. 필요한 시점에만 선택적으로 호출됩니다



Tool = AI의 외부 기능

AI Agent가 판단을 담당하고, Tool은 실제 작업을 수행합니다. 필요한 시점에만 호출됩니다



AI가 스스로 선택

여러 Tool이 연결되어 있으면 AI가 질문을 분석하여 적절한 Tool을 선택합니다



실시간 검색

SerpAPI로 웹 검색, Date&Time으로 날짜 인식을 추가할 수 있습니다

Tool의 개념과 역할

AI Agent 단독으로는 외부 데이터에 접근할 수 없습니다. Tool을 연결하면 필요한 시점에 AI가 자율적으로 호출합니다

AI Agent와 Tool의 관계

- AI Agent: 판단을 담당하는 중심 노드
- Tool: 외부 서비스나 기능을 제공하는 보조 노드(프린터, 계산기, 검색엔진 등)
- 필요한 시점에만 호출되며, 불필요한 경우 사용되지 않습니다
- Tool을 등록하더라도 항상 사용되는 것은 아닙니다(비결정적 동작)

AI Agent의 Tool 호출 흐름

- 1 사용자가 질문합니다
- 2 AI Agent가 질문을 분석합니다
- 3 필요한 Tool을 호출합니다
- 4 결과를 정리하여 답변합니다

Tool과 일반 노드의 차이

일반 노드는 순서대로 실행되지만, Tool은 AI가 필요하다고 판단할 때만 실행됩니다

구분	일반 노드	Tool 노드
연결 방식	데이터 흐름 순서대로	AI Agent에 부착 (노란색 점선)
실행 시점	순서에 따라 항상 실행	AI가 필요할 때만 실행
호출 주체	n8n 엔진이 순서대로	AI Agent가 판단하여 호출

AI가 Tool을 선택하는 기준

- 1순위: 도구의 이름(Name) - 의미가 명확해야 합니다
- 2순위: Tool Description - 용도와 발현 조건을 기술합니다
- 3순위: 파라미터 설명 - 입력 필드의 의미와 형식을 기술합니다

핵심. "my_tool_1" 같은 이름은 AI가 판단할 수 없습니다. "wikipedia_search"처럼 기능이 드러나는 이름을 사용하십시오

Wikipedia Tool로 검색 챗봇

AI Agent에 Wikipedia Tool을 연결하여 백과사전 검색 기능을 추가합니다

실습 단계

- AI Agent 하단의 + Tool 클릭합니다
- Wikipedia를 선택합니다 (별도 설정 불필요)
- Chat에서 "도커란 무엇입니까?"라고 질문합니다
- Executions에서 Tool Calls를 확인합니다

도커란 무엇입니까?

AI

도커(Docker)는 컨테이너 기반의 오픈소스 가상화 플랫폼입니다. 애플리케이션을 컨테이너라는 격리된 환경에서 실행할 수 있도록 지원합니다...

Calculator Tool 추가

Wikipedia와 함께 Calculator Tool을 연결하여 연산 기능을 추가합니다

실습 단계

- AI Agent 하단의 + Tool 클릭합니다
- Calculator를 선택합니다 (별도 설정 불필요)
- "1500원의 15% 할인 금액은?"이라고 질문합니다
- Calculator Tool이 호출되어 정확한 연산을 수행합니다

1500원짜리 상품을 15% 할인하면 얼마입니까?

AI

1500원의 15% 할인 금액을 계산하겠습니다.

$1500 \times 0.15 = 225$ 원 (할인액)

$1500 - 225 = 1,275$ 원 (최종 가격)

15% 할인 적용 시 1,275원입니다.

Multi-Tool 구성과 자동 선택

여러 Tool이 동시에 연결된 상태에서 AI가 질문 내용에 따라 적절한 Tool을 선택합니다

질문	AI 선택 Tool	이유
도커란 무엇입니까?	Wikipedia	지식 검색이 필요한 질문
1500원에서 15% 할인하면?	Calculator	수학 연산이 필요한 질문
쿠버네티스 개념을 알려주십시오	Wikipedia	기술 용어 검색
안녕하세요	호출하지 않음	Tool이 필요 없는 일반 대화

핵심. AI는 질문을 분석하여 어떤 Tool을 호출할지 스스로 판단합니다. 여러 Tool이 있어도 필요한 것만 선택하며, 필요 없으면 아무것도 호출하지 않습니다.

SerpAPI — 실시간 웹 검색

Google 검색 결과를 API로 제공하는 서비스입니다. 월 250회 무료로 사용할 수 있습니다

SerpAPI 설정 단계

- serpapi.com에서 회원 가입합니다
- Dashboard에서 API Key를 복사합니다
- AI Agent에서 + Tool 클릭 후 SerpAPI를 선택합니다
- Credential에 API Key를 붙여넣고 저장합니다
- Country(gl)를 kr, Language(hl)를 ko로 설정합니다

FREE

무료 티어

월 250회 검색 가능, 신용카드 등록이
불필요합니다



한국어 검색

Country와 Language를 한국으로 설
정하면 한국어 결과를 반환합니다

SerpAPI 검색 데모

AI Agent가 SerpAPI를 호출하여 실시간 웹 검색 결과를 반환합니다

AI

SerpAPI로 검색한 최신 AI 뉴스입니다:

1. Docker 입문 가이드 - 컨테이너 기반...
2. AI 코딩 도구 비교 - 최신 트렌드...
3. 클라우드 네이티브 전환 사례 - ...

최신 AI 뉴스 알려주세요

SerpAPI 응답 구조

json

```
{
  "organic_results": [
    {
      "title": "Docker 입문 가이드",
      "snippet": "Docker는 컨테이너...",
      "link": "https://example.com/",
      "position": 1
    }
  ]
}
```

비결정적 Tool 선택과 Max Iterations

같은 질문이라도 AI가 매번 다른 Tool을 선택할 수 있습니다

같은 질문, 다른 결과

시도	질문	AI 선택 Tool
1회차	미국 이란 전쟁 조사해 주세요	Wikipedia + SerpAPI
2회차	미국 이란 전쟁 조사해 주세요	호출하지 않음

LO-3에서 학습한 비결정적 소프트웨어의 특성이 Tool 선택에도 적용됩니다

Max Iterations 설정

설정	값	설명
기본값	10	AI가 판단할 때까지 반복
교육용 권장	3~5	Tool 반복 호출 방지

SerpAPI는 월 250회 무료입니다. Max Iterations를 설정하지 않으면 AI가 같은 Tool을 반복 호출하여 크레딧이 소진될 수 있습니다. AI Agent 노드의 Options에서 Max Iterations를 설정하십시오.

복합 검색: System Prompt로 순서 지정

System Prompt로 Tool 호출 순서를 지정하면 더 정확한 결과를 얻을 수 있습니다

```
System Prompt 예시
```

```
당신은 정확한 팩트 기반의 뉴스  
큐레이터입니다.
```

```
도구 사용 규칙:
```

```
1. 개념/용어 → Wikipedia  
2. 숫자 계산 → Calculator  
3. 실시간 뉴스 → SerpAPI  
4. 저장된 뉴스 → 뉴스DB  
5. 반복 호출 금지
```

복합 Tool 선택 패턴

질문	AI 선택 Tool
저장된 뉴스 알려주세요	뉴스DB
최신 뉴스 검색해주세요	SerpAPI
docker 뉴스	뉴스DB 또는 SerpAPI
도커의 역사와 최근 동향	Wikipedia + SerpAPI

Tool 과다 문제와 멀티 에이전트

하나의 AI Agent에 Tool 10개 이상을 연결하면 정확도가 떨어집니다

구성	Tool 수	정확도
에이전트 1개 에 Tool 15개	과다	잘못된 도구를 선택할 확률이 높아집니다
역할별 에이 전트 3개, 각 5개	적정	담당 영역의 도구만 보유하여 정확도가 향상됩니다

핵심. Tool이 많아지면 에이전트를 역할별로 분리하는 "멀티 에이전트" 패턴이 필요합니다

멀티 에이전트 패턴

검색 에이전트

Wikipedia, SerpAPI

계산 에이전트

Calculator, Date&Time

데이터 에이전트

Data Table, HTTP Request

AI Tool 활용 체크포인트

- ✓ Tool의 개념과 역할(AI가 필요한 시점에 호출하는 외부 기능)을 설명할 수 있습니다
- ✓ Wikipedia Tool과 Calculator Tool이 AI Agent에 연결되어 있습니다
- ✓ AI가 질문에 따라 적절한 Tool을 자동 선택하는 것을 확인했습니다
- ✓ SerpAPI로 실시간 웹 검색이 정상 작동합니다
- ✓ Max Iterations로 Tool 반복 호출을 제한할 수 있습니다
- ✓ Tool 과다 시 멀티 에이전트 패턴이 필요함을 이해했습니다

06

Memory 시스템

- ✓ Memory의 필요성(대화 맥락 유지)을 설명할 수 있습니다
- ✓ Simple Memory 노드를 AI Agent에 연결할 수 있습니다
- ✓ Session ID의 역할을 이해합니다
- ✓ Memory 추가 전후의 차이를 직접 확인합니다

🕒 25분

AI Agent Memory 구성

Memory가 없으면 AI Agent는 이전 대화를 유지하지 못하고 매 요청을 독립적으로 처리합니다



Memory 없는 AI

매 요청을 독립적으로 처리하여 이전 대화의 맥락을 유지하지 못합니다



Memory 있는 AI

이전 대화를 참조하여 맥락을 유지하므로 자연스러운 연속 대화가 가능합니다



Session ID

사용자별 대화를 분리하는 식별자입니다. 같은 세션끼리만 맥락이 공유됩니다

Memory 없는 AI — 맥락 부재

매 대화가 독립적으로 처리되어 이전 대화 내용을 전혀 참조하지 못합니다

AI

Docker 관련 최신 뉴스입니다.
[뉴스1, 뉴스2, 뉴스3]

docker 뉴스 알려줘

AI

무슨 뉴스를 말씀하시는 건가요?
(맥락 손실)

그 중에서 가장 중요한 거 알려줘

비교 항목	Memory 없음
"그 중에서" 이해	이해 불가
"아까 말한 것" 이해	이해 불가
대화 연속성	매번 새 대화

Memory 있는 AI — 맥락 유지

이전 대화를 참조하여 맥락을 유지하므로 연속된 대화 흐름이 자연스럽게 이어집니다

docker 뉴스 알려줘

AI

Docker 관련 최신 뉴스입니다.
[뉴스1, 뉴스2, 뉴스3]

그 중에서 가장 중요한 거 알려줘

AI

앞서 알려드린 3개 뉴스 중 가장 중요한 것은 뉴스1입니다.
(맥락 유지)

비교 항목	Memory 있음
"그 중에서" 이해	이전 답변 참조 가능
"아까 말한 것" 이해	대화 기록에서 검색
대화 연속성	자연스러운 흐름 유지

Simple Memory 노드

최근 대화를 일정 개수만큼 기억하여 AI Agent에게 전달합니다 (구 Window Buffer Memory)

Simple Memory 추가 방법

- AI Agent 노드를 클릭합니다
- 하단의 + Memory 버튼을 클릭합니다
- Simple Memory를 선택합니다
- 파란색 점선으로 연결이 표시됩니다

설정 항목	설명	기본값
Context Window Length	기억할 대화 쌍의 개수	5
Session ID Type	대화 세션 구분 방식	Auto

동작 방식. 대화가 누적되면 설정된 개수만큼만 유지하고, 가장 오래된 대화부터 삭제합니다

Session ID — 대화 세션 관리

여러 사용자의 대화를 구분하는 식별자입니다. 같은 Session ID끼리만 맥락이 공유됩니다

Session ID Type	설명	사용 사례
Auto	Chat Trigger가 자동 관리	기본 채팅 UI 사용 시
Custom	직접 Session ID 지정	Slack, Telegram 연동 시

세션 분리 예시

Session user_123

docker 뉴스 관련 대화 기록

Session user_456

kubernetes 관련 대화 기록

두 세션의 대화는 서로 독립적으로 관리됩니다

Context Window Length 선택 기준

기억할 대화 수에 따라 토큰 비용과 응답 속도가 달라집니다

Window Length	장점	단점	권장 상황
3에서 5	토큰 절약, 빠른 응답	맥락 손실 가능	단순 질의응답
5에서 10	충분한 맥락 유지	토큰 비용 증가	일반 대화 (권장)
10 이상	긴 대화 지원	비용 급증, 응답 속도 저하	복잡한 상담

이 과정에서는 기본값 5를 사용합니다. Context Window Length가 클수록 이전 대화가 더 많이 AI에게 전달되므로 토큰 사용량이 증가합니다

실습: Memory 추가 설정

이전 LO에서 만든 워크플로우에 Simple Memory를 추가합니다

설정 단계

- AI Agent 하단의 + Memory를 클릭합니다
- Simple Memory를 선택합니다
- Context Window Length를 5로 설정합니다
- Session ID Type을 Auto로 설정합니다
- 워크플로우를 저장합니다

워크플로우 구조

Chat Trigger	실선
AI Agent	실선
OpenAI Chat Model	점선
Tool (Wikipedia 등)	노란 점선
Simple Memory	파란 점선

Memory 적용 전후 비교

동일한 대화로 Memory 추가 전후의 차이를 직접 확인합니다

Before (Memory 없음)

안녕하세요, 저는 홍길동입니다

AI

안녕하세요! 무엇을 도와드릴까요?

제 이름이 뭐라고 했습니까?

AI

죄송합니다. 이름을 알려주시겠습니까?

After (Memory 추가)

안녕하세요, 저는 홍길동입니다

AI

안녕하세요, 홍길동님! 무엇을 도와드릴까요?

제 이름이 뭐라고 했습니까?

AI

홍길동이라고 말씀하셨습니다.

Memory 시스템 체크포인트

✓ Simple Memory가 AI Agent에 파란색 점선으로 연결되어 있습니다

✓ Memory 없는 상태에서 맥락 손실을 확인했습니다

✓ Memory 추가 후 이전 대화를 참조하는 응답이 생성됩니다

✓ Context Window Length의 의미를 이해했습니다

✓ Session ID의 역할을 설명할 수 있습니다

07

Google OAuth 세팅

- ✓ Google Cloud Console에서 프로젝트를 생성하고 API를 활성화할 수 있습니다
- ✓ OAuth 동의 화면을 구성하고 테스트 사용자를 등록할 수 있습니다
- ✓ OAuth 자격증명을 발급하고 n8n Credential로 등록할 수 있습니다

🕒 25분

Google OAuth 설정 가이드

n8n에서 Gmail, Calendar, Drive를 사용하기 위한 인증 설정을 완료합니다



OAuth 2.0 인증

비밀번호 없이 안전하게 접근 권한을 부여하는 표준 방식입니다



Google Cloud Console

프로젝트 생성, API 활성화, 자격증명 발급을 수행합니다



n8n Credential 등록

발급받은 자격증명으로 Gmail, Calendar, Drive를 연결합니다

OAuth 2.0의 개념과 동작 원리

비밀번호 대신 접근 토큰을 발급하여 필요한 권한만 부여하는 인증 표준입니다

호텔 카드키 비유

- 마스터키(비밀번호)를 복사하지 않습니다
- 특정 방(서비스)에만 접근 가능한 카드키(토큰)를 발급합니다
- 토큰 만료로 보안 피해를 최소화합니다
- 필요한 범위(Scope)만 선택하여 권한을 제한합니다



API 키 방식

키 유출 시 전체 노출, 권한 제어 불가



OAuth 2.0 방식

토큰 만료로 피해 최소화, 범위 선택 가능

n8n Cloud와 셀프호스팅 OAuth 차이

Cloud 환경에서는 간편 경로를, 셀프호스팅에서는 수동 설정을 진행합니다

항목	n8n Cloud	셀프호스팅
GCP 프로젝트 생성	불필요 (간편 경로)	필수
OAuth 자격 증명	n8n 자동 제공	직접 생성
리다이렉트 URI	자동 설정	수동 등록 필요
갱신 주기	Cloud 정책상 주기적 확인 권장 (공식 명문 없음)	프로덕션 게시 시 만료 없음

Google Cloud Console 프로젝트 생성

console.cloud.google.com에서 새 프로젝트를 생성합니다

프로젝트 생성 절차

- ▶ Google Cloud Console 접속합니다
- ▶ 상단 프로젝트 선택 드롭다운에서 '새 프로젝트'를 선택합니다
- ▶ 프로젝트 이름: n8n-automation
- ▶ '만들기' 선택 후 30초 대기합니다
- ▶ 상단에서 방금 만든 프로젝트가 선택되었는지 확인합니다

소요 시간: 3분

Google 계정으로 로그인한 상태에서 진행하십시오. 무료 등급으로 충분합니다.

API 3개 활성화

Gmail, Calendar, Drive API를 모두 활성화해야 이후 LO에서 각 서비스를 사용할 수 있습니다



Gmail API

이메일 읽기, 전송, 라벨 관리 기능을 제공합니다



Google Calendar API

일정 조회, 생성, 수정 기능을 제공합니다



Google Drive API

파일 업로드, 다운로드, 공유 기능을 제공합니다

→ 좌측 메뉴에서 'API 및 서비스'의 '라이브러리'를 선택합니다

→ 각 API를 검색하여 '사용' 버튼을 선택합니다

OAuth 동의 화면 구성

사용자에게 접근 권한을 승인받는 화면을 설정합니다

설정 절차

- 사용자 유형: 외부(External) 선택합니다
- 앱 이름: n8n Workflow Automation
- 사용자 지원 이메일: 본인 Gmail 선택합니다
- 테스트 사용자에게 본인 Gmail 주소를 추가합니다

주의사항

테스트 사용자에게 본인 이메일을 반드시 추가해야 합니다. 누락하면 인증 시 "Access Denied" 오류가 발생합니다.

Credential 발급 및 n8n 등록

OAuth 클라이언트 자격증명을 발급하고 n8n에 등록합니다

자격증명 발급

- '사용자 인증 정보'에서 OAuth 클라이언트 ID를 생성합니다
- 애플리케이션 유형: 웹 애플리케이션을 선택합니다
- 리다이렉트 URI: <https://app.n8n.cloud/oauth2/callback>
- Client ID와 Client Secret을 복사합니다

n8n 등록

- n8n Credentials에서 Gmail OAuth2 API를 선택합니다
- Client ID와 Secret을 붙여넣습니다
- 'Sign in with Google'을 선택합니다
- 권한 승인 후 Connected 확인합니다

팝업을 닫으면 Client Secret을 다시 볼 수 없습니다. 반드시 복사해 두십시오.

'확인되지 않은 앱' 경고 처리

테스트 모드와 프로덕션 모드의 차이를 이해하고 적절히 대응합니다

상태	토큰 만료	사용자 제한	검증 필요
테스트	7일	테스트 사용자만	불필요
프로덕션 (미검증)	만료 없음	제한 없음	경고 표시
프로덕션 (검증 완료)	만료 없음	제한 없음	Google 검증 통과

- '고급'을 선택한 후 'n8n Workflow Automation(으)로 이동'을 선택합니다
- 본인이 만든 앱이므로 안전하게 진행할 수 있습니다
- 교육 실습에는 테스트 모드로 충분합니다 (7일 후 재인증)

연결 테스트 워크플로우

3개 서비스의 Credential이 정상 동작하는지 한 번에 확인합니다

Manual Trigger

>>>

Gmail 최근 메일 1건

Calendar 이번 주 일정 1건

Drive 파일 1건 검색

1

워크플로우 импорт

lo7-01-oauth-test.json을 импорт하거나 직접 구성합니다

2

Credential 선택

각 노드에서 앞 단계에서 등록한 Credential을 선택합니다

3

Test Workflow 실행

3개 노드가 모두 녹색 체크되면 OAuth 설정 완료입니다

노드별 설정 가이드

각 노드에서 설정해야 할 항목을 확인합니다

Gmail

Resource	Message
Operation	Get Many
Limit	1

Credential만 선택하면 바로 동작합니다

Calendar

Resource	Event
Operation	Get Many
Calendar	primary
Time Min	<code>{{ \$now.toISO() }}</code>
Time Max	<code>{{ \$now.plus({ days: 7 }).toISO() }}</code>

Time Min/Max는 별표 아이콘을 눌러 Expression 모드로 전환합니다

Drive

Resource	File/Folder
Operation	Search
Limit	1

Search Query는 비워두면 전체에서 1건을 가져옵니다

Drive 노드 상세: 검색 vs ID 지정

파일을 가져오는 두 가지 방법을 비교합니다

방법 1: 이름으로 검색

설정	값
Operation	Search
Search Query	파일 또는 폴더 이름
Filter	Type을 Folder로 (폴더만 찾을 때)
Limit	1

방법 2: ID로 직접 지정

설정	값
Operation	Get
File ID	주소창의 마지막 부분 (1abc123...)

Drive URL에서 ID 찾기

```
https://drive.google.com/file/d/1abc123.../view
```

폴더 안의 파일 목록을 보려면

Operation을 List로 선택하고, Options에서 Folders to list에 폴더 ID를 입력합니다.
이 방법은 LO-10과 LO-11에서 사용합니다.

확인 결과

서비스	성공 기준
Gmail	메일 제목/발신자 표시
Calendar	일정 표시 또는 빈 배열
Drive	파일명/ID 표시

Google OAuth 세팅 체크포인트

- ✓ Google Cloud Console에서 프로젝트를 생성했습니다
- ✓ Gmail, Calendar, Drive API 3개를 모두 활성화했습니다
- ✓ OAuth 동의 화면을 구성하고 테스트 사용자를 등록했습니다
- ✓ OAuth 클라이언트 자격증명을 발급했습니다
- ✓ n8n에서 Credential을 등록하고 연결 테스트를 통과했습니다
- ✓ 테스트 워크플로우에서 Gmail, Calendar, Drive 데이터를 확인했습니다

08

AI 이메일 분류

- ✓ AI Agent에 Gmail Tool을 연결하여 이메일을 읽고 조작할 수 있습니다
- ✓ System Prompt로 이메일 분류 기준을 정의할 수 있습니다
- ✓ AI가 이메일을 분석하여 자동으로 라벨을 적용하는 워크플로우를 구축합니다

🕒 30분

AI 이메일 분류 자동화

AI Agent가 이메일을 읽고 분석하여 자동으로 라벨을 부여하는 워크플로우를 구축합니다



Gmail Tool 연결

AI Agent가 Gmail을 자율적으로 읽고 조작할 수 있게 합니다



라벨 자동 분류

AI가 이메일 내용을 분석하여 적절한 라벨을 적용합니다



프롬프트 설계

System Prompt로 분류 기준과 행동 규칙을 정의합니다

이메일 분류 자동화 시나리오

기존 IF 노드 방식과 AI 방식의 차이를 비교합니다

항목	기존 방식 (IF 노드)	AI 방식 (Agent + Gmail Tool)
분류 기준	키워드, 발신자 등 수동 규칙	AI가 내용을 이해하여 판단
확장	새 규칙마다 노드 추가	프롬프트 수정만으로 가능
복잡한 판단	어려움	맥락과 뉘앙스 이해 가능
요약	불가능	AI가 이메일 요약 가능

AI 분류는 규칙을 코드로 만들 필요 없이, 프롬프트 한 줄 수정으로 분류 기준을 변경할 수 있습니다.

Gmail Tool 노드의 역할

AI Agent가 Gmail을 자율적으로 읽고, 라벨을 생성하고, 할당할 수 있게 해주는 도구입니다

구분	일반 Gmail 노드	Gmail Tool 노드
동작 방식	개발자가 동작을 직접 지정	AI Agent가 상황 판단 후 결정
연결 대상	워크플로우 흐름	AI Agent 노드
유연성	고정된 규칙	프롬프트로 유연하게 변경
적합한 경우	단순 반복 작업	판단이 필요한 복잡한 작업

AI Agent에 Gmail Tool 연결

Gmail Tool의 Operation과 Simplify 옵션을 설정합니다

주요 Operation

Operation	설명
Get Many	여러 이메일을 목록으로 조회합니다
Get	특정 이메일 1건의 전체 내용을 조회합니다
Add Label	이메일에 라벨을 적용합니다
Create Label	새로운 라벨을 생성합니다

Simplify 옵션

- Get Many: Simplify ON (Snippet만 전달, 토큰 절약)
- Get: Simplify OFF (본문 전체 데이터 필요)
- Limit/Search를 By Model로 설정하면 AI가 자율 결정합니다

라벨 사전 생성 및 자동 부여

Gmail에 분류용 라벨을 미리 생성하고, AI가 자동으로 할당하도록 합니다



긴급

제목이나 본문에 긴급, 즉시, ASAP이 포함된 이메일



고객문의

고객, 문의, 견적, 구매 관련 이메일



업무협조

보고서, 회의, 일정, 프로젝트 관련 이메일



뉴스레터

뉴스, 구독, newsletter, 홍보 관련 이메일

Gmail에서 라벨을 사전에 생성해 두어야 합니다. 라벨이 존재하지 않으면 AI가 라벨 적용 시 오류를 반환합니다.

System Prompt 설계

역할 정의, 분류 기준, 행동 규칙 세 가지를 명확히 포함합니다

좋은 프롬프트의 3요소

- 역할 정의: '이메일 분류 비서'로 역할을 명확히 합니다
- 분류 기준: 4개 카테고리 and 판단 키워드를 나열합니다
- 행동 규칙: 분류 후 라벨 적용까지 지시합니다

당신은 이메일 분류 비서입니다.
읽지 않은 이메일을 확인하고
다음 4개 카테고리로 분류하십시오:

1. 긴급
2. 고객문의
3. 업무협조
4. 뉴스레터

각 이메일에 분류 결과와 이유를
알려주십시오.
분류 후 해당 라벨을 Gmail에
적용하십시오.

Gmail Trigger로 자동화 전환

새 이메일이 도착하면 폴링 주기에 따라 분류가 자동 실행됩니다

항목	Chat Trigger	Gmail Trigger
실행 방식	사용자가 직접 명령 입력	폴링 주기마다 새 메일 확인 후 실행
폴링 간격	해당 없음	최소 1분 (Webhook 아님)
사용 시점	프로토타이핑 및 테스트	프로덕션 운영
활성화	수동 실행	워크플로우 Active 상태 필수



Chat Trigger (테스트)

사용자가 명령을 입력하면 분류를 실행합니다



Gmail Trigger (프로덕션)

폴링(최소 1분)으로 새 메일을 확인하여 자동 분류합니다

전체 워크플로우 구조

Chat Trigger에서 AI Agent, Gmail Tool까지의 전체 연결을 확인합니다



Chat Trigger

사용자 명령을 수신합니다



AI Agent

이메일 분석 및 분류를 판단합니다



OpenAI Chat Model

gpt-5-nano로 분류 작업을 수행합니다



Gmail Tool

이메일 조회 및 라벨 적용을 수행합니다

Chat Trigger

>>>

AI Agent

>>>

Gmail Tool

테스트 및 디버깅

n8n 내장 Chat 창으로 분류 결과를 테스트하고 조정합니다

테스트 절차

- Gmail에 테스트용 이메일 2~3개를 전송합니다
- n8n Chat 창에서 '최근 이메일을 분류해 주세요'를 입력합니다
- AI Agent의 동작(조회, 분석, 라벨 할당)을 관찰합니다
- Gmail에서 라벨이 실제로 적용되었는지 확인합니다

분류 기준 조정

- 분류 결과가 의도와 다르면 System Prompt를 수정합니다
- '마케팅 이메일도 뉴스레터로 분류하십시오' 등 규칙 추가
- 수정 후 다시 테스트하여 결과를 비교합니다

프로토타이핑에서 프로덕션으로

AI Agent의 역할을 판단 로직만으로 축소하여 비용과 지연 시간을 줄입니다

단계	AI가 담당하는 범위	일반 노드가 담당하는 범위
프로토타입	메일 조회, 내용 분석, 라벨 적용 전체	없음
프로덕션	분류 판단만 수행	메일 조회, 라벨 적용 (Gmail 노드)

초기에는 AI Agent로 빠르게 검증하고, 안정화 후에는 AI의 역할을 판단 로직만으로 축소하는 것이 실무 패턴입니다.

AI 이메일 분류 체크포인트

- ✓ AI Agent에 Gmail Tool을 연결하고 Operation을 설정했습니다
- ✓ Gmail에 분류용 라벨을 사전에 생성했습니다
- ✓ System Prompt로 분류 기준을 정의했습니다
- ✓ AI가 이메일을 읽고 자동으로 라벨을 적용하는 것을 확인했습니다
- ✓ 분류 기준을 수정하고 결과가 달라지는 것을 확인했습니다

09

외부 노출 — ngrok 터널

- ✓ 셀프호스팅 n8n을 외부에 노출해야 하는 이유 이해
- ✓ ngrok 대시보드에서 Authtoken 및 무료 고정 도메인 발급
- ✓ .env 두 줄 작성만으로 n8n과 ngrok 동시 기동
- ✓ WEBHOOK_URL과 N8N_EDITOR_BASE_URL 자동 주입 확인
- ✓ 터널 재기동 시에도 도메인이 유지됨을 확인

🕒 20분 · 블록 6 전반

localhost로 처리할 수 없는 두 가지 시나리오

외부 서버가 로컬 n8n으로 직접 POST 요청을 전송해야 하는 경우입니다



외부 서비스 Webhook 수신

[Discord](#) · [Slack](#) · [Stripe](#) → [나의 Webhook](#)

외부 서비스가 이벤트를 n8n 으로 보내려면 외부에서 접근 가능한 주소가 필요합니다. localhost 는 외부 서버가 도달할 수 없습니다



Google Push Notification 수신

[Gmail](#) · [Drive](#) · [Calendar](#) → [나의 Webhook](#)

메일 수신, 파일 변경, 일정 추가 같은 이벤트를 실시간으로 받으려면 Google 서버가 직접 POST 요청을 보낼 공개 HTTPS 주소가 필요합니다. localhost 는 도달 불가능합니다

핵심. 외부에서 도달 가능한 HTTPS 주소가 필요합니다 — 그래서 터널이 등장합니다

본 과정의 터널 선택 — ngrok 무료 고정 도메인

별도 도메인 보유 없이 고정 도메인을 발급받을 수 있는 가장 간단한 구성입니다

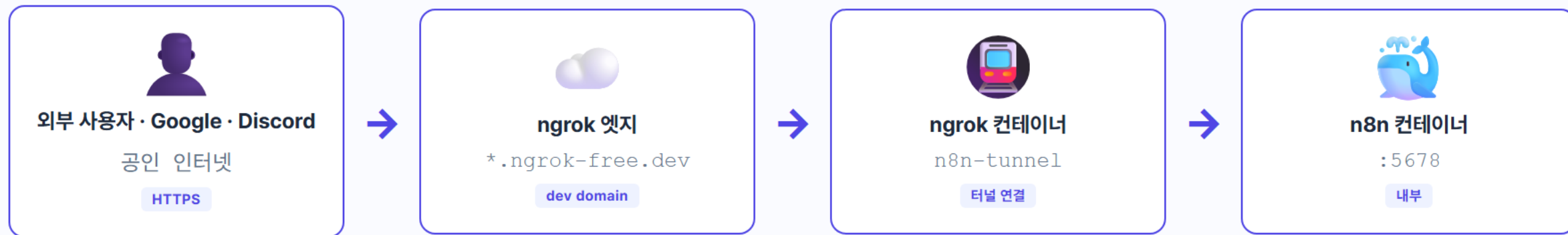
옵션	무료	도메인 고정	본인 도메인	수업 적합도
ngrok (Free)	○	○	불필요	★★★ 권장
Cloudflare Quick Tunnel	○	× 매번 변경	불필요	★★
Cloudflare Named Tunnel	○	○	필수	★★ (보유 시)
localhost.run · Serveo	○	△ 불안정	불필요	★

옛 자료의 **n8n start --tunnel** 명령은 사라졌습니다. n8n 1.x 에 있던 자체 터널 기능은 v2.0 에서 제거되었으며 현재는 ngrok · Cloudflare Tunnel · localtunnel 같은 외부 서비스를 사용해야 합니다.

도메인 고정의 의미. cloudflared Quick Tunnel 처럼 URL 이 매번 바뀌지 않습니다. .env 에 한 번 기록해 두면 끄고 켜도 같은 URL 을 그대로 씁니다. OAuth redirect URI 등록도 한 번이면 끝입니다.

외부 요청이 n8n까지 도달하는 경로

네 단계 모두 단일 docker-compose 구성 내에서 관리됩니다



compose 내부 네트워크에서는 ngrok 컨테이너가 서비스 이름 **n8n** 만으로 컨테이너에 도달합니다 (host.docker.internal 불필요)

ngrok 대시보드에서 발급받는 두 값

무료 가입 후 발급되며, 본 과정 전체에서 동일한 값을 재사용합니다

Authtoken

1

dashboard.ngrok.com/get-started/your-authtoken

ngrok 클라이언트가 본인 계정으로 인증합니다. 한 번 발급 후 .env 의 NGROK_AUTHTOKEN 에 넣습니다

dev domain (고정 URL)

2

dashboard.ngrok.com/domains → Create Domain

*.ngrok-free.dev 형태 (2026-01 리브랜딩, 기존 계정은 .app 유지). 무료 계정당 dev domain 1개. 한 번 만들면 영구적으로 본인 것이며 .env 의 TUNNEL_DOMAIN 에 넣습니다

무료 플랜 제약. 동시 활성 endpoint 최대 3개, dev domain 1개, 1GB/월 대역폭, 20k HTTP 요청/월, 최초 접속 시 안내 페이지 1회 표시 등이 있으나, 모두 수업 시연에는 영향이 없습니다. 강사가 사전에 한 번 Visit Site를 클릭하면 동일 브라우저에서는 재노출되지 않습니다.

ngrok 서비스 블록 추가 구성

LO-2의 docker-compose 파일에 본 블록을 추가하고 .env에 두 줄을 작성하면 외부 노출이 완료됩니다

```
# docker-compose.yml - ngrok 서비스 발췌
ngrok:
  image: ngrok/ngrok:latest
  container_name: n8n-tunnel
  restart: unless-stopped
  depends_on: [n8n]
  environment:
    - NGROK_AUTHTOKEN=${NGROK_AUTHTOKEN}
  command: http --url=${TUNNEL_DOMAIN} n8n:5678
# 요청 로그(http://localhost:4040)가 필요하면 아래 두 줄 주석 해제
# ports:
#   - "4040:4040"
```

강의 레포 / docker-compose.yml

command 한 줄이 하는 일

발급받은 고정 도메인을 사용해 같은 네트워크 안의 n8n:5678 으로 모든 외부 트래픽을 전달

compose 의 서비스 이름 매직

같은 docker compose 안의 서비스는 이름만으로 서로 도달 가능 —
host.docker.internal 불필요

환경변수 자동 주입 흐름

.env 한 곳에서만 도메인을 관리하면, docker-compose가 모든 위치로 자동 전파합니다

.env (한 번만 작성)

```
# .env (강의 레포 폴더 안에 작성)
NGROK_AUTHTOKEN=2abc ... 본인토큰
TUNNEL_DOMAIN=n8n-yourname.ngrok-free.dev
```



n8n 컨테이너 env (자동 주입)

```
# n8n 환경변수 - compose 가 ${TUNNEL_DOMAIN} 자동 치환
N8N_HOST=${TUNNEL_DOMAIN}
N8N_PROTOCOL=https
WEBHOOK_URL=https://${TUNNEL_DOMAIN}
N8N_EDITOR_BASE_URL=https://${TUNNEL_DOMAIN}
N8N_PROXY_HOPS=1
```

검증 명령. `docker exec n8n env | grep -E 'WEBHOOK_URL|N8N_HOST'` — 출력에 ngrok 도메인이 들어 있어야 정상

ngrok 터널 실행 및 n8n 연결

7단계로 진행되며, Authtoken만 발급되어 있으면 .env 두 줄로 구성이 완료됩니다

- 1 Authtoken · 고정 도메인 발급** ngrok 대시보드 두 페이지에서 복사 (3 min)
- 2 .env 두 줄 작성** NGROK_AUTHTOKEN, TUNNEL_DOMAIN — 강의 레포 폴더 안 (2 min)
- 3 컨테이너 기동** docker compose up -d → n8n + ngrok 함께 (2 min)
- 4 외부 URL 접속** 브라우저에서 https://본인도메인 → 안내 페이지 통과 (2 min)
- 5 env 주입 확인** docker exec n8n env | grep WEBHOOK_URL (2 min)
- 6 Webhook URL 확인** 신규 워크플로우 + Webhook 노드 → Production URL 확인 (2 min)
- 7 OAuth redirect URI 추가 (선택)** Google Cloud Console 에 ngrok callback URL 등록 (2 min)

ngrok 환경에서 자주 발생하는 오류

에러 코드별 핵심 해결책을 정리하였으며, 로그를 참조하여 대응합니다

증상	원인	해결
ERR_NGROK_4018 (auth failed)	NGROK_AUTHTOKEN 미설정 또는 오타	.env 의 토큰 재확인 후 ngrok 컨테이너 재생성
ERR_NGROK_3200 (domain not found)	TUNNEL_DOMAIN 이 대시보드와 불일치	Domains 페이지에서 정확한 도메인 다시 복사
ERR_NGROK_108 (endpoint limit)	무료 플랜은 동시 활성 endpoint 최대 3개. 다른 세션의 endpoint가 누적되어 한도 초과	다른 세션 종료 또는 Endpoints 에서 stale 제거
외부 URL 접속 시 502	n8n 컨테이너가 안 떠 있거나 5678 포트 막힘	docker ps · docker logs n8n 로 부팅 로그 확인
Webhook URL 이 'https://' 만 표시	.env 의 TUNNEL_DOMAIN 비어 있음 (compose 경고 출력)	.env 두 변수 확인 후 docker compose up -d --force-recreate n8n
Chat Trigger 만 안 됨 (Invalid origin)	N8N_PUSH_BACKEND=sse 가 설정되어 EventSource 가 거부됨	해당 env 제거 (기본값 WebSocket 사용), 브라우저 하드 새로고침
OAuth redirect_uri_mismatch	Google Console 에 ngrok URI 미등록	Console 에 ngrok callback URL 한 줄 추가 등록

외부 노출(ngrok) 체크포인트

✓ ngrok 대시보드에서 Authtoken 과 dev domain(고정 URL) 발급 완료

✓ .env 에 NGROK_AUTHTOKEN, TUNNEL_DOMAIN 두 줄 기록

✓ docker compose up -d 한 줄로 n8n 과 ngrok 함께 기동

✓ https://본인도메인.ngrok-free.dev 으로 브라우저 접속 성공

✓ docker exec n8n env 에 ngrok 도메인이 들어 있음을 확인

✓ n8n 워크플로우의 Webhook URL 이 ngrok 도메인으로 표시됨

✓ 터널을 끄고 켜도 도메인이 같게 유지됨을 이해

10

Discord 연동

- ✓ Discord Webhook을 생성하고 메시지를 전송할 수 있습니다
- ✓ Webhook Trigger로 외부 요청을 수신할 수 있습니다
- ✓ 외부 메신저 연동 패턴을 이해하고 적용할 수 있습니다

🕒 25분

Discord 연동 자동화

n8n 워크플로우의 결과를 Discord 채널로 전송하고, 외부 요청을 수신하는 패턴을 학습합니다



Discord Webhook

URL로 메시지를 전송하여 채널에 알림을 보냅니다



Webhook Trigger

외부 시스템에서 n8n 워크플로우를 호출합니다



외부 메신저 패턴

Chat UI에서 실서비스 메신저로 전환하는 방법을 학습합니다

외부 메신저 연동 패턴

n8n Chat UI에서 실서비스 메신저로 전환하는 구조를 이해합니다



n8n Chat UI (테스트용)

Chat Trigger로 n8n 내부에서 테스트합니다



외부 메신저 연동 (실서비스)

Webhook으로 요청을 받고 Discord로 결과를 전송합니다

메신저	연동 방법
Discord	Discord Webhook 노드
Slack	Slack Webhook 노드
Teams	Microsoft Teams Webhook
Telegram	Telegram Bot API

Discord Webhook 개념

특정 URL로 데이터를 POST하면 지정된 채널에 메시지가 올라가는 방식입니다

Webhook이란

- URL로 메시지를 전송하는 방식입니다
- 별도의 Bot 계정이 필요하지 않습니다
- 채널별로 독립된 Webhook URL을 생성합니다
- URL이 노출되면 누구나 메시지를 보낼 수 있으므로 보관에 주의합니다

Webhook URL 형식:

```
https://discord.com/api/webhooks/  
[WEBHOOK_ID]/[WEBHOOK_TOKEN]
```

POST 요청 시 해당 채널에
메시지가 자동으로 게시됩니다.

Discord Webhook 생성 방법

Discord 채널 설정에서 Webhook URL을 발급합니다

생성 절차

- Discord에서 실습 채널(#n8n-test)을 생성합니다
- 채널 이름 옆 설정(톱니바퀴) 아이콘을 선택합니다
- 좌측 메뉴에서 '연동'을 선택합니다
- '웹훅 보기'를 선택합니다
- '새 웹훅'을 선택하고 이름을 'N8N Bot'으로 설정합니다
- '웹훅 URL 복사'를 선택하여 URL을 저장합니다

보안 주의사항

Webhook URL은 비밀번호처럼 보관하십시오. 노출되면 누구나 해당 채널에 메시지를 보낼 수 있습니다.

n8n Discord 노드 설정

AI Agent의 출력을 Discord 채널로 전송하도록 노드를 설정합니다

설정 항목	값	설명
Authentication	Webhook	웹훅 방식으로 인증합니다
Webhook URL	(복사한 URL)	Discord에서 생성한 Webhook URL
Content	{{ \$json.output }}	AI Agent의 응답을 전달합니다

- AI Agent 뒤에 Discord 노드를 추가합니다
- Webhook URL에 복사한 URL을 붙여넣습니다
- Content에 AI Agent 응답 표현식을 입력합니다

Webhook Trigger로 외부 요청 수신

Chat Trigger 대신 Webhook Trigger를 사용하여 외부 시스템에서 워크플로우를 호출합니다

항목	Chat Trigger	Webhook Trigger
용도	n8n 채팅 UI	외부 시스템 연동
입력 형식	고정 (chatInput)	자유 (body)
URL	자동 생성	자동 생성
활성화	수동 실행	Active 상태 필수

- Webhook Trigger 노드를 클릭하여 Production URL을 복사합니다
- 워크플로우를 저장하고 Active 상태로 전환합니다
- 요청 body는 {{ \$json.body.content.text }}로 접근합니다

전체 아키텍처

Webhook 수신에서 AI 처리, Discord 전송까지의 전체 흐름입니다

Webhook Trigger

>>>

AI Agent

>>>

Discord Webhook

- n8n trigger 앱이 Discord 메시지를 감지하여 Webhook으로 전달합니다
- AI Agent가 메시지를 분석하고 응답을 생성합니다
- Discord Webhook으로 AI 응답을 채널에 전송합니다

cURL 테스트 실행

Webhook URL로 POST 요청을 보내고 Discord 채널에서 결과를 확인합니다

```
curl -X POST \  
  "https://[n8n-url]/webhook/[path]" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "content": {  
      "text": "AI 뉴스 검색해줘"  
    }  
  }'
```

결과 확인

- Discord 채널에 AI 응답 메시지가 표시됩니다
- n8n Executions에서 실행 기록을 확인합니다
- 워크플로우가 Active 상태인지 반드시 확인합니다

양방향 연동: n8n trigger 앱

수강생이 봇을 직접 개발하지 않고, 제3자 운영 trigger 앱이 Discord와 n8n을 연결합니다

Discord 메시지

>>>

n8n trigger 앱

>>>

n8n Webhook

>>>

AI Agent

>>>

Discord Webhook

1

앱 추가

n8n trigger 디스코드 앱(앱 ID 1389933424331980993)을 서버에 추가합니다. 수강생이 봇을 직접 개발하지는 않습니다

2

Webhook 연결

n8n Webhook의 Production URL을 /setup 명령으로 등록합니다

3

메시지 전송

채널에 메시지를 보내면 AI가 처리하여 같은 채널에 응답합니다

운영 주의 — 제3자 봇 의존성. n8n trigger 앱은 n8n 공식이 아닌 제3자가 운영하는 Discord 봇을 빌려 쓰는 구조입니다. 봇 서버가 중단되면 양방향 흐름도 함께 멈춥니다. 프로덕션에 서는 본인 Discord 봇 직접 구현 또는 `n8n-nodes-discord-trigger` 커뮤니티 노드 설치를 검토하십시오.

양방향 설정 가이드

3단계로 Discord AI 챗봇을 완성합니다

STEP 1 1 Discord 설정

- 1 n8n trigger 디스코드 앱 링크에 접속합니다
- 2 '앱 추가' 버튼을 클릭합니다
- 3 봇을 추가할 서버를 선택하고 승인합니다

STEP 2 2 n8n 설정

- 1 lo9-02-discord-bot.json을 임포트합니다
- 2 OpenAI, Discord Webhook Credential을 설정합니다
- 3 워크플로우를 저장하고 Active로 전환합니다
- 4 Webhook 노드의 Production URL을 복사합니다

STEP 3 3 연결

- 1 Discord 채널에서 /setup 명령을 입력합니다
- 2 Production URL을 붙여넣습니다
- 3 'Webhook Setup Complete' 메시지를 확인합니다
- 4 /enable 명령으로 활성화합니다

Discord Developer Portal에서 봇을 직접 만들 필요가 없습니다. n8n trigger 앱이 메시지 중개 역할을 대신합니다.

워크플로우 구조 비교

단방향과 양방향의 차이는 트리거뿐입니다

lo9-01: 단방향

- 1 Chat Trigger
- 2 AI Agent
- 3 Discord Webhook

입력 소스 n8n 채팅창

Discord 앱 불필요

n8n 채팅창에서 입력하면 AI가 응답하고 Discord 채널에 전송합니다

lo9-02: 양방향

- 1 Webhook (POST)
- 2 AI Agent
- 3 Discord Webhook

입력 소스 Discord 채널 메시지

Discord 앱 n8n trigger 앱 추가

Discord 채널에서 메시지를 보내면 n8n trigger 앱이 Webhook으로 전달하고, AI 응답이 같은 채널에 표시됩니다

Discord 연동 체크포인트

- ✓ Discord 채널에 Webhook을 생성하고 n8n에서 메시지를 전송했습니다
- ✓ n8n trigger 디스코드 앱을 서버에 추가했습니다
- ✓ Webhook URL을 /setup으로 등록하고 양방향 통신이 동작합니다
- ✓ 단방향과 양방향 연동의 차이를 설명할 수 있습니다

11

Drive 기반 RAG 시스템

- ✓ 임베딩의 개념과 벡터 데이터베이스의 역할을 설명할 수 있습니다
- ✓ Google Drive의 PDF를 청크 분할하여 Pinecone에 임베딩 저장할 수 있습니다
- ✓ RAG 원리를 이해하고 Vector Store QA Tool 기반 문서 검색 챗봇을 완성할 수 있습니다

🕒 45분

Drive 기반 RAG 시스템 구축

Google Drive의 PDF를 임베딩하여 Pinecone에 저장하고, 문서 기반 AI 챗봇을 완성합니다



Drive 문서 수집

Google Drive에서 PDF를 가져와 Extract from File로 본문을 추출합니다



Pinecone 임베딩 저장

청크 분할 + OpenAI Embeddings로 벡터화하여 Pinecone 인덱스에 저장합니다



RAG 챗봇

AI Agent + Vector Store QA Tool로 저장된 문서를 검색하여 답변합니다

RAG의 개념과 동작 원리

Retrieval-Augmented Generation: 외부 참고자료를 검색하여 LLM 응답에 결합하는 기술입니다



핵심 비유

일반 AI는 암기 시험, RAG AI는 오픈북 시험을 보는 것과 같습니다

AI 모델을 직접 학습시키지 않고도, 우리 조직의 문서를 기반으로 정확한 답변을 제공할 수 있습니다



1단계: 검색

사용자 질문과 관련된 문서 조각을 벡터 DB에서 찾습니다



2단계: 증강

검색된 문서를 AI Agent의 컨텍스트에 추가합니다



3단계: 생성

문서 기반으로 정확한 답변을 생성합니다

일반 AI와 RAG AI 비교

RAG를 적용하면 조직 내부 데이터를 기반으로 정확한 답변이 가능합니다

질문 예시	일반 AI	RAG AI
우리 회사 휴가 규정 알려줘	일반적인 정보만 답변합니다	회사 규정집에서 찾아 정확히 답변합니다
프로젝트 A 진행 상황은?	알 수 없다고 답변합니다	사내 문서 검색 후 답변합니다
작년 매출은?	추측하거나 거절합니다	보고서 기반으로 정확한 수치를 제공합니다

RAG의 가치. AI 모델을 직접 학습시키지 않고도 조직 문서를 근거로 답변할 수 있어, 추가 학습 비용 없이 도메인 지식을 결합할 수 있습니다.

주의. 검색 품질이 답변 품질을 결정합니다. 청킹 크기와 임베딩 모델 선택이 최종 답변의 정확도를 좌우합니다.

임베딩(Embedding) 개념

텍스트를 숫자 배열(벡터)로 변환하여 의미 기반 검색을 가능하게 합니다

지도 비유

- 서울과 인천은 지도에서 가까이 위치합니다
- 서울과 부산은 지도에서 멀리 위치합니다
- 임베딩도 마찬가지입니다: 의미가 비슷하면 가까운 벡터를 갖습니다
- 텍스트를 '의미 지도'에 배치하는 것입니다

벡터 변환 예시

"강아지" [0.23, -0.45, 0.67, ...]

"개" [0.21, -0.43, 0.65, ...]

가까운 벡터

"자동차" [-0.12, 0.89, -0.34, ...]

먼 벡터

벡터 데이터베이스: Pinecone

임베딩된 벡터를 저장하고 유사도 검색을 수행하는 전문 데이터베이스입니다

일반 데이터베이스	벡터 데이터베이스
정확한 키워드로 검색합니다	의미가 비슷한 것으로 검색합니다
"휴가"만 찾습니다	"연차", "쉬는 날"도 찾습니다
SQL 쿼리를 사용합니다	벡터 유사도를 계산합니다



Pinecone

클라우드 기반 벡터 DB로, n8n에서 바로 연결할 수 있습니다



폐쇄망 대안

Qdrant, Chroma 등 오픈소스 벡터 DB를 로컬에 설치할 수 있습니다

Pinecone 인덱스 생성 — 1536 차원 필수

인덱스 차원과 임베딩 모델 차원이 일치해야 합니다. 불일치 시 노드가 무한 로딩 상태에 빠집니다

인덱스 생성 설정

- Pinecone 콘솔에서 Create Index를 클릭합니다
- Dimensions를 반드시 1536으로 설정합니다
- Metric은 cosine을 선택합니다
- n8n에서 Pinecone API Credential을 등록합니다

모델 차원 매칭

모델	차원	인덱스 차원
text-embedding-3-small	1536	1536 필수
text-embedding-3-large	3072	3072 필수

차원 불일치 시 Pinecone 노드가 응답 없이 멈춥니다. 실습 시 가장 흔한 실수입니다.

문서 청킹 (Chunking)

긴 문서를 적절한 크기로 분할하여 임베딩합니다

- 문서 전체를 임베딩하면 검색 정확도가 떨어집니다
- 책 전체가 아닌 관련 페이지만 참조하는 방식과 유사합니다
- AI 모델의 토큰 한도를 초과할 수 있습니다
- Recursive Character Text Splitter로 분할합니다

본 실습 설정. Chunk Size 200 / Overlap 50 — 빠른 반복 학습과 작은 PDF 시연을 위한 **교육용 미니멈**입니다. 운영 환경 검색 품질로는 부적합하니, 실제 RAG 시스템 구축 시 운영 권장값(약 500~512 토큰 + Overlap 10~20%) 범위에서 자료 특성에 맞게 튜닝하십시오.

설정	설명	교육 실 습	운영 권장
Chunk Size	조각의 최대 크기 (문자 수)	200	500~512 토큰 (한국어 약 700~900자)
Chunk Overlap	인접 조각 간 겹치는 부분	50	Chunk Size의 10~20%
Separators	분할 기준 문자	\n\n, \n, 공백	동일

두 개의 워크플로우로 구성되는 RAG 시스템

저장 워크플로우와 질의 워크플로우는 독립적으로 동작하며, Pinecone 인덱스를 공유하여 연결됩니다

사전 1회 실행

LO11-03. 문서 임베딩 저장

지식 인덱스를 구축하는 준비 단계 · 진입점 Manual Trigger

- 1 Drive Search · Download
- 2 Extract PDF (텍스트 추출)
- 3 Default Data Loader · Text Splitter
- 4 OpenAI Embeddings → Pinecone Insert

질문이 들어올 때마다 실행

LO11-04. RAG 챗봇

사용자 질문에 응답하는 운영 단계 · 진입점 Chat Trigger

- 1 AI Agent + OpenAI Chat Model
- 2 Vector Store QA Tool 호출
- 3 Pinecone Retrieve (유사도 검색)
- 4 검색 결과를 근거로 답변 생성

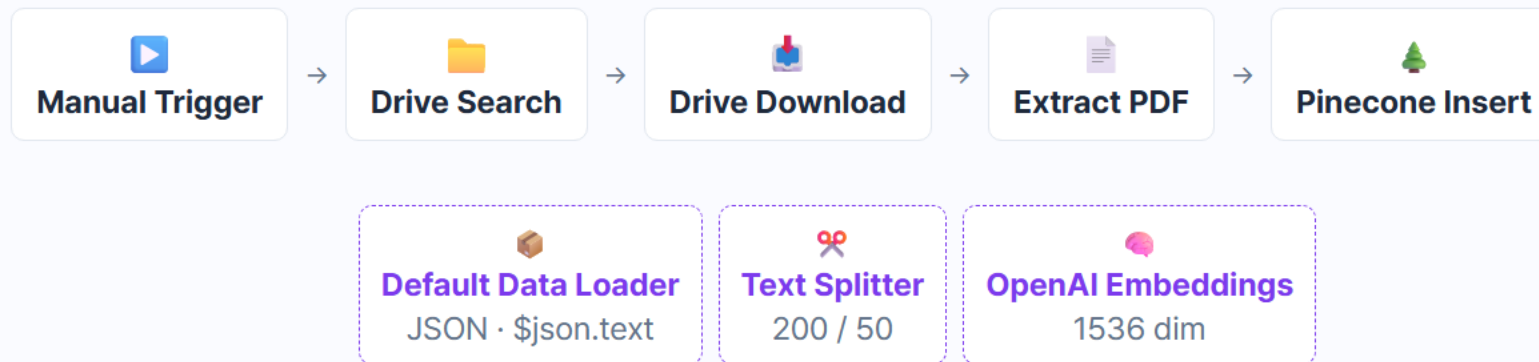


연결고리 — Pinecone Vector DB (공유 인덱스)

저장 워크플로우가 적재한 벡터를 질의 워크플로우가 동일 인덱스에서 검색하므로, 두 워크플로우는 시간적으로 분리되어도 데이터로 연결됩니다.

Drive → Pinecone 임베딩 파이프라인

Google Drive의 PDF를 다운로드하여 텍스트를 추출하고 Pinecone에 임베딩 저장합니다



Default Data Loader — 무의미한 청크 제거

PDF 메타데이터(버전, 작성자 등)가 임베딩되지 않도록 본문 텍스트만 추출합니다

Data Loader 설정

설정	값
Type of Data	JSON
Mode	Load Specific Data
JSON Data	{{ \$json.text }}
Text Splitting	Custom

Load Specific Data로 설정해야 본문만 임베딩됩니다

설정 차이로 인한 결과

설정	저장되는 청크
기본값 (전체 입력)	"1.4", "False", "anonymous" 포함
Load Specific Data	본문 텍스트만 깨끗하게

무의미한 청크가 포함되면 검색 품질이 저하되므로, 반드시 \$json.text만 추출해야 합니다

RAG 챗봇 워크플로우 구성

Chat Trigger와 AI Agent를 연결하고, Vector Store Tool로 문서를 검색합니다

구성 순서

- Chat Trigger와 AI Agent를 연결합니다
- OpenAI Chat Model을 AI Agent에 연결합니다
- System Message에 문서 검색 도우미 역할을 지정합니다
- Vector Store QA Tool을 Tool 슬롯에 추가합니다



Chat Trigger

사용자의 질문을 입력받습니다



AI Agent

문서를 검색하고 답변을 생성합니다



Vector Store QA Tool

Pinecone에서 관련 문서를 검색합니다

Vector Store QA Tool — 서브노드 두 개 필수

QA Tool은 Vector Store와 Model을 모두 연결해야 합니다. Model 누락 시 sub-node 에러가 발생합니다

Tool 파라미터

설정	값
Name	company_docs
Description	회사 내부 문서를 검색합니다
Top K	3

Name은 System Prompt의 Tool 호출명과 반드시 일치해야 합니다

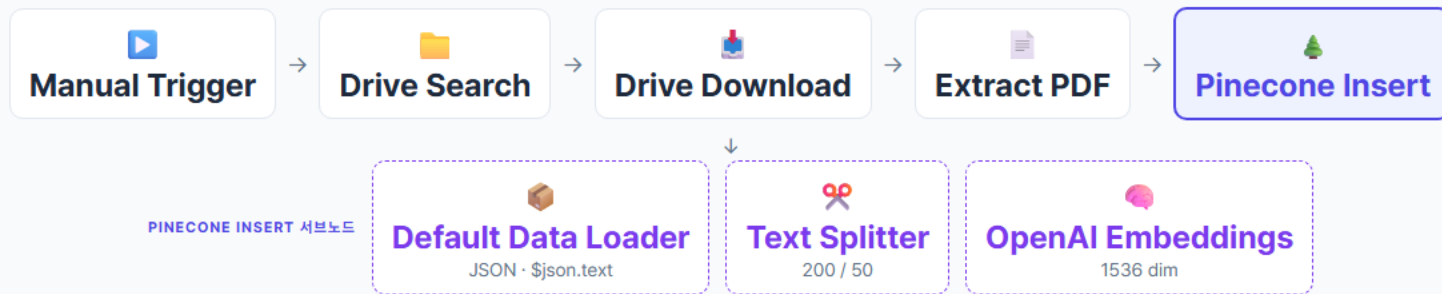
서브노드 연결

- Model 슬롯 → OpenAI Chat Model (청크 요약용 LLM)
- Vector Store 슬롯 → Pinecone Vector Store (Retrieve)
- Pinecone 하단 → Embeddings OpenAI (저장 시와 동일 모델)
- Model이 누락되면 sub-node 에러가 발생합니다

전체 RAG 시스템 아키텍처

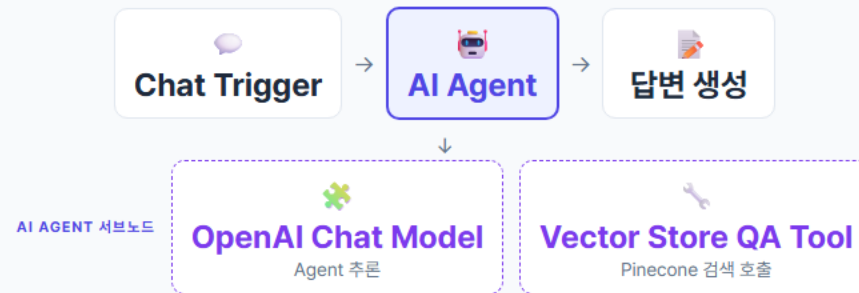
저장 스택과 질의 스택이 Pinecone 인덱스를 공유하여 연결됩니다

STORE LO11-03. 문서 임베딩 저장 (사전 1회 실행)



Pinecone Vector DB — 공유 인덱스로 두 스택을 연결

QUERY LO11-04. RAG 챗봇 (질문 발생 시 실행)



Drive 기반 RAG 시스템 체크포인트

- ✓ 임베딩을 '텍스트를 지도에 배치하는 것'으로 설명할 수 있습니다
- ✓ Pinecone 인덱스를 1536 차원으로 생성하고 Credential을 등록했습니다
- ✓ Google Drive의 PDF를 Extract from File로 텍스트 추출했습니다
- ✓ Default Data Loader를 Load Specific Data + \$json.text로 설정하여 메타데이터를 차단했습니다
- ✓ Vector Store QA Tool에 Vector Store와 Model 두 서브노드를 모두 연결했습니다
- ✓ AI Agent가 company_docs 도구를 호출하여 문서 기반 답변을 생성하는 것을 확인했습니다

12

과정 마무리

- ✓ 11개 학습 목표 완주를 확인합니다
- ✓ 다음 학습 방향과 실무 적용 아이디어를 정리합니다
- ✓ 수강 후 Q&A 와 연락 방법을 안내합니다

🕒 3분

11개 학습 목표 완주

셀프호스팅 n8n부터 RAG 챗봇까지, 실무에 곧바로 적용 가능한 자동화 스택을 완성했습니다

01 n8n 기초 개념

워크플로우, 트리거, 노드, 아이템, 실행

02 Docker 셀프호스팅

compose 기동, 운영 명령, 영속 데이터

03 데이터 흐름

표현식, IF·Set·Merge, 실행 로그

04 AI Agent 기초

Chat Trigger, OpenAI Chat Model, System Prompt

05 AI Tool 활용

Wikipedia·Calculator·SerpAPI, Multi-Tool

06 Memory 시스템

Simple Memory, Session ID, Context Window

07 Google OAuth 세팅

Cloud Console, 동의 화면, Credential 등록

08 AI 이메일 분류

Gmail Tool, 라벨 자동화, Gmail Trigger

09 ngrok 터널

고정 도메인, .env 주입, OAuth redirect

10 Discord 연동

Webhook 송수신, 양방향 챗봇

11 Drive 기반 RAG

임베딩, Pinecone, Vector Store QA

Trigger와 Action 한 줄 연결에서 시작해, AI Agent · Tool · Memory · RAG까지 완성했습니다.

다음 단계 — 실무 적용과 계속 학습

배운 패턴을 실제 업무에 옮기는 방법과 지속적으로 따라잡을 자료를 정리했습니다



실무 PoC 시작

본인 업무 중 반복 작업 한 가지를 골라 Chat Trigger-AI Agent로 프로토타입을 만듭니다



프로덕션 전환

AI 역할을 판단 로직만으로 축소하고, 일반 노드 (Gmail·Sheets·HTTP)로 행동을 대체합니다



팀에 공유

워크플로우 JSON을 사내 레포에 보관하고, 강의 레포 구조처럼 시나리오별로 분리합니다

추가 학습 자료

공식 문서 docs.n8n.io
템플릿 갤러리 n8n.io/workflows
커뮤니티 community.n8n.io
릴리즈 노트 github.com/n8n-io/n8n/releases

n8n 버전 업그레이드 대응

- 이미지 태그를 :latest 대신 명시적 버전(예: 2.20.9)으로 고정합니다
- 릴리즈 노트에서 Breaking Changes 항목을 우선 확인합니다
- 운영 환경 업그레이드 전 별도 컨테이너로 회귀 테스트를 수행합니다

Thank you

끝까지 함께해 주셔서 감사합니다

CONTACT

강사	조나래
소속	(주) 드림플로우
이메일	info@dream-flow.com